Pricing and Hedging of Barrier Options

Inès El Krissi and Stanislas Romagny

Research Thesis Supervisor:
Prof. Vincent Milhau, EDHEC Business School

June 2025

Table of Content

1 Introduction.....3

2 Theoretical Background.....4

2.1 Barrier Options.....4

2.2 Monte Carlo.....7

2.3 Black-Scholes-Merton Model.....8

3. Volatility Models 11

3.1 Flat Volatility Assumption.....11

3.2 Local Volatility Model.....12

4 Pricing Method..….....13

4.1 Monte Carlo Simulations.....13

4.2 Finite Difference Method 13

4.2.1 Boyle & Tian Method. 13

4.2.2 Crank- Nicolson for Barrier Options.....14

4.3 Implementation and Comparisons.....16

5. Greeks.....18

5.1 Delta.....18

5.2 Gamma.....19

5.3 Theta.....20

5.4 Vega.....22

5.5 Rho.....23

6. Hedging.....24

6.1 Delta Hedging.....24

6.2 Boundary Replication.....29

Conclusions.....34

Bibliography.....35

Appendix….....36

Introduction:

Derivatives markets have become an essential part of the global the financial system netting almost to $800 trillion in 2024. Growth in trading activity has been particularly fast in the over-the-counter markets thanks to their ability to offer exotic derivatives customized to investors' needs. Among the most successful OTC options appear barrier options. The payoff of a barrier option depends on whether the price of the underlying crosses a predefined threshold before maturity. We encounter two types of barrier options, knock-in and knock-out options. Knock-in options begin to exist when the price of the underlying reaches the trigger level whereas knock-out option is rendered worthless if the price of the underlying touches the barrier.

Barrier options allow market participants to tailor their trading strategies to their specific market views. Investors who believe in a limited upward move of the price of the underlying will prefer to buy an up-and-out calls rather than more expensive plain vanilla calls. The premium of a barrier option is lower than that of a vanilla option because its value is derived from a more restricted set of price paths compared to a vanilla option of the same type. The holder of a barrier option forfeits part of the traditional value of the option, which results in a lower trading price. On the other hand, the seller of a barrier option reduces their overall risk exposure compared to a vanilla option. The lower price of barrier options makes them one of the most popular exotic options. They offer speculators the opportunity to achieve greater leverage for the same amount invested.

Barrier options are complex instruments to hedge as they combine features of plain vanilla products and a bet on whether the underlying price might or might not reach a trigger level. Indeed, adding a barrier impact significantly, the sensitivity of the option value to changes in the underlying price, volatility, time to maturity or interest rates. Therefore, the Greeks of a barrier option behave very differently from the ones of a plain vanilla option. The discontinuity in the payoff of barrier options complexifies the process of hedging them, especially those that are in the money when they come out of existence such as up-and-out calls and down-and-out puts. For instance, when the price of the underlying asset trades close to the barrier and option is about to expire, the Delta and the Gamma of an up and out call take large negative values because the value of the option vary greatly in this region. Vega of the option also becomes negative near the barrier as an increase in volatility increases the likelihood of the price of the underlying crossing the barrier level. In contrast, the Delta, Gamma and Vega of a vanilla call are all positive with no extreme fluctuations. We will further discuss mathematical models allowing us to price these financial instruments thanks to the principle of replication.

2. Theoretical Background

2.1 Barrier Options:

Let us define S(T) price of the underlying asset at maturity, K the strike price and B the barrier level, then the payoffs of up and out call and down and out put are given by:

Up-and-In Call:

$$\max(S_T - K, 0) \times \mathbf{1}_{\exists t \in [0,T], S_t \geq B}$$

Down-and-In Put:

$$\max(K - S_T, 0) \times \mathbf{1}_{\exists t \in [0,T], S_t \leq B}$$

The payoffs for the up-and-out call and down-and-out put are:

Up-and-Out Call:

$$\max(S_T - K, 0) \times \mathbf{1}_{S_t < B, \forall t \in [0,T]}$$

Down-and-Out Put:

$$\max(K - S_T, 0) \times \mathbf{1}_{S_t > B, \forall t \in [0,T]}$$

The **call-put parity relationship** for vanilla option is written as,

$$C_t - P_t = S_t - Kexp(-r(T-t))$$

where C_t is the Call price at date t of exercise price K, of maturity T, P_t is the Put price at date t with same parameters, and S_t is the price of the underlying asset at date t.

This relationship encounters difficulties regarding barrier options because its decomposition $(S_T - K)_+ - (K - S_T)_+ = S_T - K$, must be multiplied by $1_{\underset{0 \leq t \leq T}{max} S_t \leq B}$, where B is the barrier level. $S_T . 1_{\underset{0 \leq t \leq T}{max} S_t \leq B}$, does not correspond to any classical financial instrument.

However, when summing two barrier options together, “les indicatrices” of opposite events, $1_{\underset{0\le t\le T}{max}\, S_t \le B} + 1_{\underset{0\le t\le T}{max}\, S_t \ge B} = 1$. A portfolio consisting of one knock-in call, and one knock out call is equivalent to a classical call option,

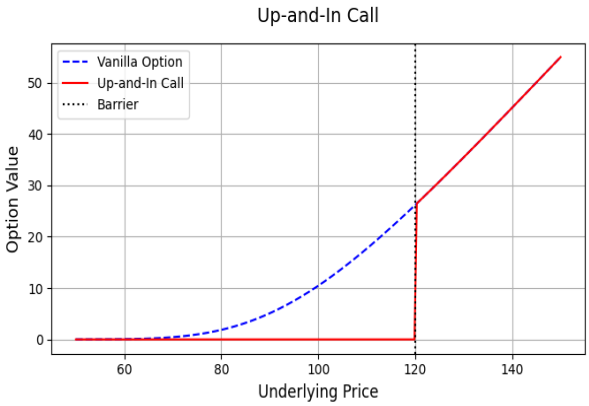
Up-and-Out Call + Up-and-In Call = Vanilla Call

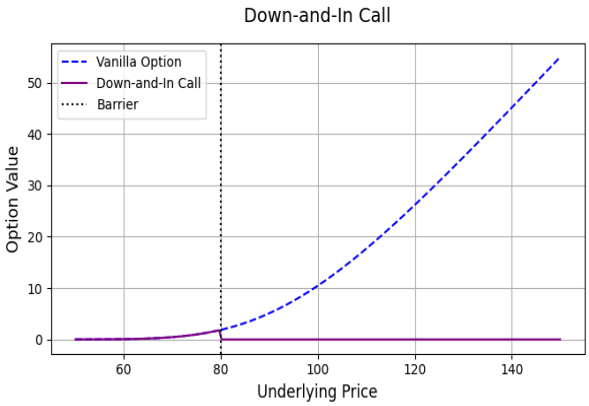
Similarly, for other barrier options combination we have the following relationships,

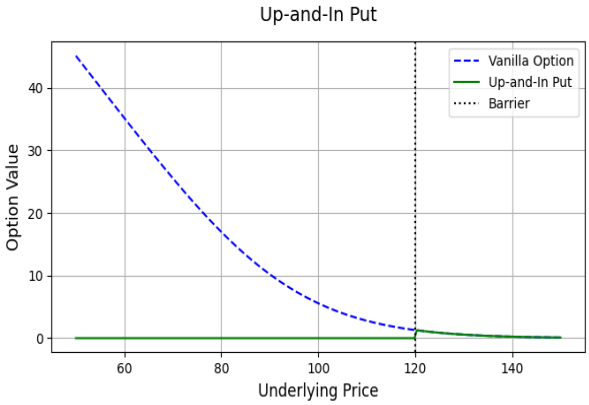
Down-and-Out Call + Down-and-In Call = Vanilla Call

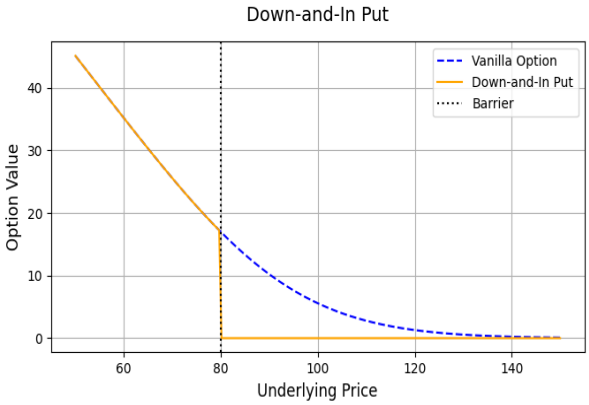
Up-and-Out Put + Up-and-In Put = Vanilla Put

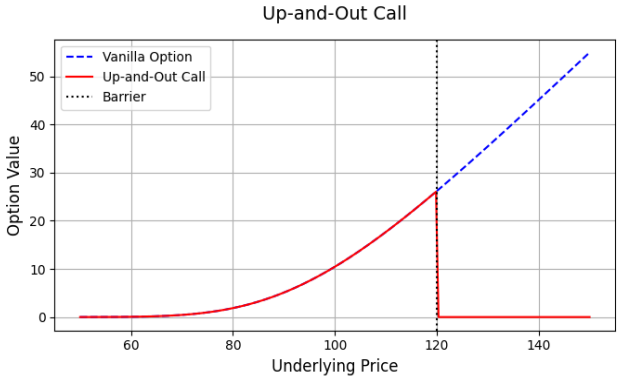
Down-and-Out Put + Down-and-In Put = Vanilla Put

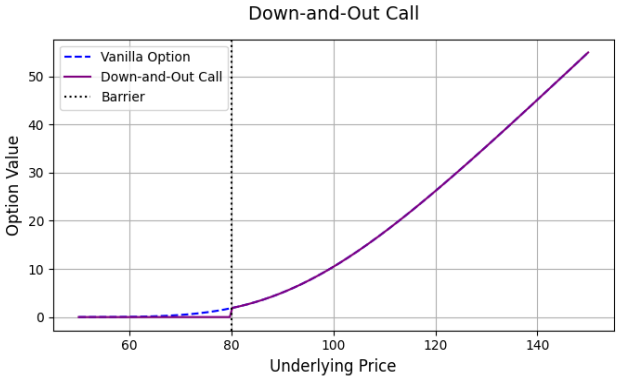

Up-and-In Call
Vanilla Option
Up-and-In Call
Barrier
Option Value
Underlying Price

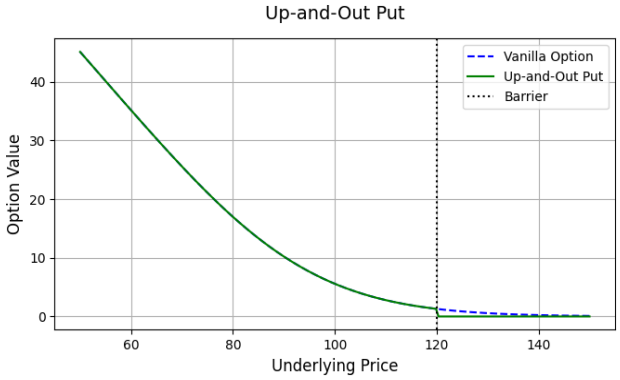

Down-and-In Call
Vanilla Option
Down-and-In Call
Barrier
Option Value
Underlying Price

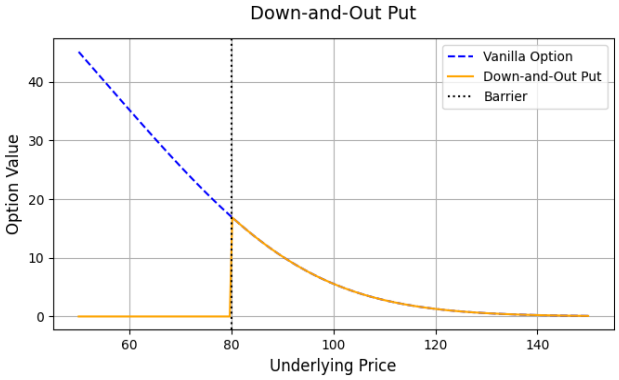

Up-and-In Put
Vanilla Option
Up-and-In Put
Barrier
Option Value
Underlying Price


Down-and-In Put
Vanilla Option
Down-and-In Put
Barrier
Option Value
Underlying Price


Up-and-Out Call
Vanilla Option
Up-and-Out Call
Barrier
Option Value
Underlying Price


Down-and-Out Call
Vanilla Option
Down-and-Out Call
Barrier
Option Value
Underlying Price


Up-and-Out Put
Vanilla Option
Up-and-Out Put
Barrier
Option Value
Underlying Price


Down-and-Out Put
Vanilla Option
Down-and-Out Put
Barrier
Option Value
Underlying Price

2.2 Pricing using Monte Carlo:

We are trying to estimate the expected value $E(X)$ of a real random variable of finite variance $Var(X)$. We simulate many independent variables $(X_i)_{i\leq N}$ such that,

$$\overline{X} = \frac{1}{N}\sum_{i=0}^{N} X_i \approx E(X)$$

For sufficiently large N, the empirical mean almost surely converges to $E(X)$ according to the law of large numbers.

The average approximation error is controlled thanks to the central limit theorem giving us for sufficiently large N, with 95% probability,

$$E(X) \in \left[\overline{X_N} - 1.96\sqrt{\frac{var(X)}{N}}, \overline{X_N} + 1.96\sqrt{\frac{var(X)}{N}} \right]$$

where variance of X can be estimated with usual estimators.

Application

In the Black-Scholes Model we have,

$$S_T = S_0 exp\left[\left(r - \frac{\sigma^2}{2}\right)T + \sigma W_T\right]$$

Where W_T follows a normal distribution $N(0, T)$.

2.3 The Black-Scholes Model :

The Black-Scholes Model is model for pricing derivatives securities where the price of the option depends on the parameters : the underlying's price S, the strike price K, the barrier level B (in the case of barrier options specifically), the interest rate r, and the underlying's volatility σ.

We aim to derive the valuation formula for barrier option in a Black-Scholes environment. More specifically it implies that the stock evolution follows a log-normal distribution with constant volatility. We assume that there are no transaction costs and a perfectly liquid market. Moreover, we suppose continuous trading and interest rates as well as dividend yields are constant variables through the life of the option. These assumptions, even if quite restrictive and not reflecting real market conditions, closely model the real-world market.

Definition (Wiener Process): A wiener process also known as Brownian motion is a continuous-time stochastic process W_t , $t \geq 0$ satisfying the following properties:

(i) Continuity of path

(ii) For any $t > 0, W_T \sim N(0,t)$, and for $0 \leq s \leq t, W_t - W_s$ is normally distributed with mean 0 and variance $t - s$, that is $W_t - W_s \sim N(0, t-s)$.

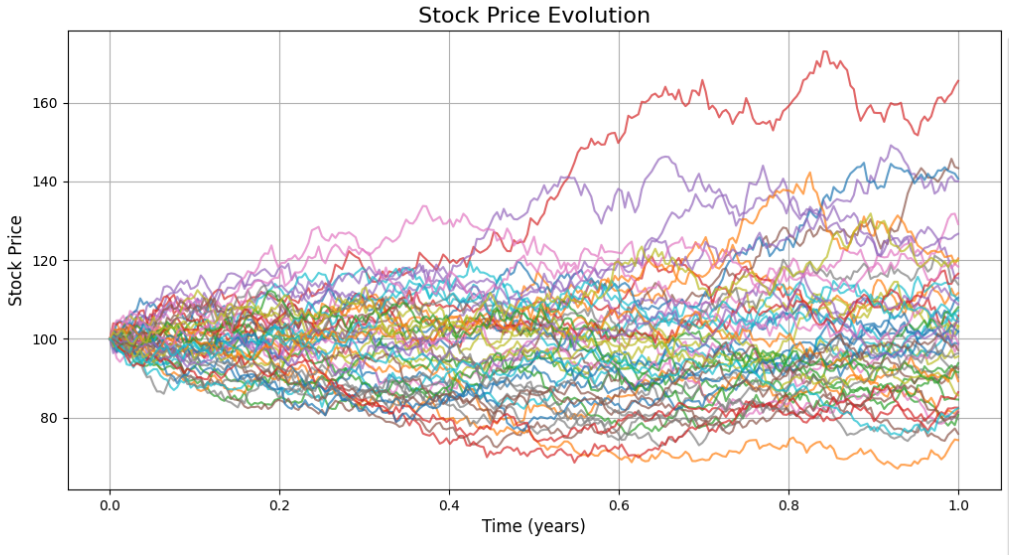
(iii) Independence of increments : For any choice of $t_i \in [0,T]$ with $0 < t_1 < t_2 < \cdots < t_n$, the increments $W_{tn} - W_{t_{n-1}}, W_{t_{n-1}} - Wt_{n-2,} \ldots W_{t_2} - W_{t1}$ and W_{ti} are independent.

Consider a non-paying dividend stock with constant volatility, σ and a constant risk-free interest rate, r. The stock price evolution is defined as:

$$\frac{\partial S_t}{S_t} = rdt + \sigma dW_t \qquad (1)$$

Where W_t is the Brownian Motion, S_t is the stock price and dS_t is the change in stock price at time t. The solution to the equation is,

$$S_t = S_0 exp\left[\left(r - \frac{\sigma^2}{2}\right)t + \sigma W_t\right]$$



Lemma (Itô's Formula) : Given a twice- differentiable function $F(S_t, t)$ of S_t and t where S_t follows a stochastic differential equation $dS_t = adt + \sigma_t dW_t$, then

$$dF_t = \frac{\partial F}{\partial S_t} dS_t + \frac{\partial F}{\partial t} dt + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 F}{\partial S_t^2} dt$$

$$= \left[\frac{\partial F}{\partial S_t} a_t + \frac{\partial F}{\partial t} + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 F}{\partial S_t^2} \right] dt + \frac{\partial F}{\partial S_t} \sigma_t dW_t \qquad (2)$$

Options are written against assets whose values depends on the underlying asset's price. Because the asset price evolution is stochastic, the option value $V(S, t)$ satisfies Ito's formula. Suppose constant risk-free rate and volatility we get,

$$dV = \frac{\partial V}{\partial S} dS + \frac{\partial V}{\partial t} dt + \frac{1}{2}\sigma^2 S_t^2 \frac{\partial^2 V}{\partial S^2} dt$$

From Itô lemma, the Wiener process underlying $V(S, t)$ and S are the same. In other words, W_t defined in (1) and (2) are the same. Therefore, the portfolio composed of one position in the derivative and one position in the underlying stock can be constructed to eliminate the Wiener process.

$$Portfolio := \{ \ -1 \ derivative \ ; \ + \frac{\partial V}{\partial S} \ shares \ \}$$

The holder of the portfolio is short one derivative and long an amount $\frac{\partial V}{\partial S}$ of shares.

Define π the value of the portfolio such that,

$$\pi(S,t) = -V(S,t) + \frac{\partial V}{\partial S}S \qquad (3)$$

Changes $\Delta\pi$ in the value of the portfolio during Δt are,

$$\Delta\pi(S,t) = -\Delta V(S,t) + \frac{\partial V}{\partial S}\Delta S$$

It follows from equations (1) and (2) that

$$d\pi(S,t) = -dV(S,t) + \frac{\partial V}{\partial S}dS$$

$$= \left(-\frac{\partial V}{\partial t} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}\right)dt. \qquad (4)$$

Because this equation does not involve any stochastic process, the portfolio must be riskless during dt and earn the same return as short-term risk-free securities. If it earned more arbitrageurs could borrow money to buy the portfolio. If it earned less, arbitrageurs could short the portfolio and buy risk-free securities.

It follows that,

$$\Delta\pi(S,t) = r\pi\Delta t \qquad (5)$$

Using equation (3) and (4) , equation (5) becomes

$$\left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}\right)\Delta t = r\left(V - \frac{\partial V}{\partial S}\Delta S\right)\Delta t$$

So that,

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} = rV \qquad (6)$$

The price of any derivative at any instant should satisfy this last equation under the Black Scholes Model assumptions and certain boundary conditions. For instance, the price $\pi(t) = V(S_t, t)$ of an Up-and-Out barrier option satisfies equation (6) with boundary conditions

$$V(B,t) = 0 \quad \text{for } tT \; knock-out \; at \; the \; barrier$$

$$V(S_T,T) = g(S_T) \; payoff \; at \; maturity$$

3.Pricing and Volatility Models

3.1 Flat Volatility Assumption.

The Black-Scholes valuation offers a closed-form formula for the pricing of barrier options, at the cost of a constant volatility assumption that may not be realistic and can lead to mispricing.

For each barrier option type, the prices are the following (taken from "*Exotic Options & Hybrids – A guide to structuring, pricing and trading, Mohammed Bouzoubaa, Adel Osseiran ,2010*) :

Up-&-In Call Price = $S\mathcal{N}(x1)e^{-qT} - Ke^{-rT}\mathcal{N}\left(x_1 - \sigma\sqrt{T}\right) - Se^{-qT}\left(\frac{H}{S}\right)^{2\lambda}[\mathcal{N}(-y) - \mathcal{N}(-y_1)] + Ke^{-rT}\left(\frac{H}{S}\right)^{2\lambda-2}\left[\mathcal{N}\left(-y + \sigma\sqrt{T}\right) - \mathcal{N}(-y_1 + \sigma\sqrt{T})\right]$

With $x_1 = \frac{ln(S/H)}{\sigma\sqrt{T}} + \lambda\sigma\sqrt{T}$, $y_1 = \frac{ln(\frac{H}{S})}{\sigma\sqrt{T}} + \lambda\sigma\sqrt{T}$, $\lambda = \frac{r-q+\sigma^2/2}{\sigma^2}$, $y = \frac{ln[\frac{H^2}{SK}]}{\sigma\sqrt{T}} + \lambda\sigma\sqrt{T}$

From the parity relationship we get: Up-&-Out Call Price = Call(K,T) – Up-&-In Call(K, T, H).

Down-&-In Put Price = $-S\mathcal{N}(-x_1)e^{-qT} + Ke^{-rT}\mathcal{N}\left(-x_1 + \sigma\sqrt{T}\right) + Se^{-qT}\left(\frac{H}{S}\right)^{2\lambda}[\mathcal{N}(y) - \mathcal{N}(y_1)] - Ke^{-rT}\left(\frac{H}{S}\right)^{2\lambda-2}\left[\mathcal{N}\left(y - \sigma\sqrt{T}\right) - \mathcal{N}(y_1 - \sigma\sqrt{T})\right]$

And from the parity relationship we get: Down-&-Out Put Price (*K, T, H*) = Put(*K,T*) – Down-&-In Put(*K, T, H*).

However, as suggested earlier, the flat volatility assumption is not realistic and will lead to mispricing in the case of barrier options.

Indeed, while the Black-Scholes model suppose the volatility of an underlying to be constant over time, and the same for all strikes and maturities, it is not the case on the derivatives market : we can see that the volatility plotted against strikes will have the shape of a smile, or more precisely a "smirk'' on the equity derivatives market, with low-strike options (ie OTM

put and ITM call) exhibiting higher implied volatility than higher-strike options (ie ITM put and OTM call).

The volatility skew is a matter that must be considered while pricing the barrier options due to the presence of a distinct strike and barrier that represent two thresholds for the payoff of the option. As we will see in more detail in the Hedging Section, the volatility around the barrier has a crucial influence on the behavior of the payoff: for a down and out put option, a higher volatility near the barrier will increase the probability of hitting the barrier and extinct the option. Thus, a volatility skew that give higher volatility for price level at the barrier will have for effect to reduce the price of the option.

3.2 The Local Volatility Model

A good manner of integrating the skew to the pricing method is to use a local volatility model.

To understand local volatility, we first need to consider that the presence of a skew indicate that the underlying doesn't follow a log-normal distribution, and the market of option is attributing to the random behavior of the underlying is due to some yet unknown distribution. The issue is now the retrieve this distribution from the option prices we are observing on the market, ie find the exact distribution that would explain all the option prices observed on the market. This is exactly what Local Volatility do, and more specifically the Dupire's local volatility model.

The local volatility model allows to capture the skew by putting the volatility of the underlying as a deterministic function of the time and the level of the underlying.

In the previous explanation about assets prices behavior and Black-Scholes pricing method, we used to denote the underlying's volatility with $\sigma\ constant.$ now we will write it as $\sigma = \sigma(S(t), t)$. We want to be able to express in a closed form this function $S(t), t \rightarrow \sigma(S(t), t)$, and the process to get to it is the "*calibration''*.

Knowing the market prices of vanilla option, the Dupire's formula gives us the volatility at any time and asset price's level:

$\sigma(K, t) = \sqrt{2\frac{\frac{\partial C}{\partial T} + (r-q)K\frac{\partial C}{\partial K} + qC}{K^2\frac{\partial^2 C}{\partial K^2}}}$ with C = C(K, T) the price of a K-strike, T years-maturity Call, r the riskless interest rate for the required period, et q dividend rate.

Calibrating a local volatility model will allow us to determine for each price level what is the relevant volatility level as an input into our pricing methods, allowing us to have a price for our barrier options that is more realistic and that account for the skew.

4. Pricing Method

4.1 Monte-Carlo Simulations

Monte-Carlo method is a powerful pricing instrument that can be adapted very quickly to any kind of derivatives. On this matter, it is an obvious choice for the pricing of barrier options.

Its basic principle has already been developed in introduction.

Basically, we will simulate a large number of price paths taken randomly and attribute a payoff function defined by the boundary and terminal conditions that fit our type of barrier option, to take the discounted average of the resulting payoffs. It will be our option's price.

The interest of this method is that it will allow us to implement volatility models through the modelization of the underlying price's behavior.

However, the Monte-Carlo method suffer from a major drawback: it requires a lot of computation time. This can be a real issue when we need to simulate a large number of scenarios to be sure to get a price that reflect all the possible outcome of the underlying asset.

4.2 The Finite Difference method

The Finite Difference Method (FDM) is an alternative to the basic Monte-Carlo simulation, allowing to obtain an accurate approximation of the option's price with a minimum computational time.

Its aims to approximate the partial differential of a function by a combination of its already known values at specific points.

In this document we will consider two different FDM: the Boyle & Tian Explicit Finite Difference approach, and an adaptation to rebate barrier option pricing of Crank-Nicolson Finite Difference method.

4.2.1 Boyle & Tian Method:

The Boyle & Tian method aims to build a trinomial tree of some discrete possible outcome of the asset's price and run backward calculation that allows to compute the barrier option's value at each time step knowing its value at the three possible price level in the following time step. That way, we are able to start from the already known intrinsic value at maturity (ie payoff of the option) and end up with the price of the derivative.

This method is not limited to the pricing of barrier option but can be adapted to it to provide better estimation.

We start from the known Black-Scholes PDE:

$$\frac{\partial f}{\partial t} + rS\frac{\partial f}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 f}{\partial S^2} = rf \ .$$

we then apply the transformation to S(t) : $y(t) = log\big(S(t)\big)$.

We end up with the following PDE: $\frac{\partial f}{\partial t} + \left(r - \frac{\sigma^2}{2}\right)\frac{\partial f}{\partial y} + \frac{1}{2}\sigma^2\frac{\partial^2 f}{\partial y^2} = rf$. (1)

The idea is then to build a 2-dimension evenly spaced grid in time and price level : a x-axis starting at t_0 the current time and ending at T the time to maturity (t_0, $t_0 + \Delta t, \dots, T$), and a y-axis starting at $y_0 = \log S_0$ and expending around this starting point at regular interval (y_0, $y_0 + \Delta y \dots$).

The matter is now about the position of the barrier regarding the grid. If the barrier level is located between 2 points of the price levels, it forces to adjust the Δy steps to force it to fit in, but at the cost of possible numerical instability and slow convergence. The solution is to set the grid in order for it to include from the start the barrier level among its values.

We can then at each node of the three approximate the partial derivatives by putting :

$\frac{\partial f}{\partial t} \approx \frac{f(i+1,j)-f(i,j)}{\Delta t}$,

$\frac{\partial f}{\partial y} \approx \frac{f(i+1,j+1)-f(i+1,j-1)}{2\Delta y}$,

$\frac{\partial f}{\partial y} \approx \frac{f(i+1,j+1)+f(i+1,j-1)-2f(i+1,j)}{\Delta y^2}$.

Where $f(i,j) = f(t_0 + i\Delta t, y_0 \pm i\Delta t)$

Substituting these three approximations in (1) give :

$$f(i,j) = \frac{1}{1+r\Delta t}[p_d f(i+1,j-1) + p_n f(i+1,j) + p_u f(i+1,j+1)]$$

With $p_u = \frac{1}{2\lambda^2} + \frac{(r-\frac{\sigma^2}{2})\sqrt{\Delta t}}{2\lambda\sigma}$, $p_n = 1 - \frac{1}{\lambda^2}$, $p_d = \frac{\sigma^2\Delta t}{2\Delta y^2} - \frac{(r-\frac{\sigma^2}{2})\sqrt{\Delta t}}{2\lambda\sigma}$, acting like probabilities, and $\lambda = \sqrt{1{,}5}$.

This value is chosen for stability and convergence speed reasons when putting $\Delta y = \lambda\sigma\sqrt{\Delta t}$ while building the grid.

The Method used here is called “explicit” because it give directly the value of the option at time t knowing it possible values at time t+1 in a unique and direct expression.

Now we must define the terminal and boundary conditions for our options that will determine the payoff at maturity and the behavior of option value across time.

In the case of a down-and-out barrier option, it will be:

$f(N,j) = max\{S(N,j) - K, 0\}\ for\ any\ j = 1, \dots, m$, and

$f(i,0) = 0 \; for \; i = 0, \dots, N$

To finally compute our option price, we will need to proceed to an adjustment regarding the scaling of the grid: we made sure that the barrier level is part of the grid, but the starting asset price has no reason to do the same.

This force us to adjust the grid so that it starts at time $t_0 - 1$: it will ensure that the starting price is actually at the middle of the 3 possibilities (up, neutral, down) of the first step of the new grid. A simple quadratic interpolation will then allow to obtain accurately this final price knowing its value at the three nodes of the time step.

4.2.2 Crank-Nicolson for barrier options

The Crank-Nicolson scheme works on the same principle than for previous studied finite difference method but benefits from more numerical stability and convergence speed thanks to a combination of explicit and implicit method.

By using the same discretization in time and price level, we build a grid of values of assets at discrete time steps : a x-axis made of the values of $t = 0, \Delta t, \dots, n\Delta t = T$, and a y-axis made of the values of $S = 0, \Delta S, \dots, m\Delta S = S_{sup}$. At each node the value of the option can be denoted $V_{i,k} = V(t_i, S_k)$ with $S_k = k\Delta S$.

The specificity of the Crank-Nicolson Finite Difference Method is that it will make the same discretization of the Black-Scholes PDE than we did in the Tian & Boyle method, but using different point of the node (backward and forward), and then the average of the backward and forward difference method on the same boundary conditions.

$\frac{\partial f}{\partial t} + rS\frac{\partial f}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 f}{\partial S^2} = rf$.

A forward discretization method will give us :

$\frac{V_{i,k} - V_{i-1,k}}{\Delta t} + rk\Delta S\left[\frac{V_{i-1,k+1} - V_{i-1,k-1}}{2\Delta S}\right] + \frac{(\sigma k \Delta S)^2}{2}\left[\frac{V_{i-1,k+1} - 2V_{i-1,k} + V_{i-1,k-1}}{\Delta S^2}\right] = rV_{i-1,k}$

A backward discretization method will give us :

$\frac{V_{i,k} - V_{i-1,k}}{\Delta t} + rk\Delta S\left[\frac{V_{i,k+1} - V_{i,k-1}}{2\Delta S}\right] + \frac{(\sigma k \Delta S)^2}{2}\left[\frac{V_{i-1,k+1} - 2V_{i,k} + V_{i,k-1}}{\Delta S^2}\right] = rV_{i,k}$

By taking the average of two methods we obtain:

$$-\lambda_k V_{i-1,k-1} - (1+\beta_k)V_{i-1,k} - \eta_k V_{i-1,k+1} = \lambda_k V_{i,k-1} + (-1+\beta_k)V_{i,k} + \eta_k V_{i,k+1}$$

With $\lambda_k = \frac{\Delta t}{4}(rk - \sigma^2 k^2)$, $\beta_k = \frac{\Delta t}{2}(\sigma^2 k^2 + r)$, $\eta_k = \frac{-\Delta t}{4}(rk + \sigma^2 k^2)$

This result in a tridiagonal matrix that is solvable to give us the final price of the option by running backward calculation starting from payoffs at time T.

4.3 Implementation and Comparison

We will use as benchmark the closed-form Black-Scholes price for down-and-in put, which is 2.968398 for S0 = 100, K = 100, B = 80 and a volatility of 17.9596%.

Using the Monte-Carlo simulation method with local volatility implementation we end up with a price for a down-and-in put of 3.585229 with 1 000 000 simulations at 252 time steps. The same simulation with flat volatility return a price of 2.8449.

Using the Crank-Nicolson Implicit Finite Difference Method the computation yields a put down-and-in barrier option price of 2.9549. The option being priced is a European put option with a down-and-in barrier. This means that the option becomes active only if the underlying asset price falls below a specified barrier (in this case, 85) during its life. If the barrier is not breached, the option expires worthless. The current spot price of the underlying asset is 90, which is close to the barrier, indicating a moderate probability of activation, especially given the volatility of 17.9596% and the time to maturity of 1 year.

Using the Boyle and Tian method, the obtained results show that the call knock-out option is priced at approximately 9.64, which is reasonable given that the underlying asset starts at 100 and the barrier is set at 90. This means there is a moderate chance that the option remains alive until maturity, thus justifying a non-negligible premium. The call knock-in option is priced significantly lower, at around 0.0415, which aligns with the intuition that knock-in options only gain value if the barrier is hit during the option's life. This relationship respects the fundamental parity where the vanilla option price equals the sum of the knock-in and knock-out prices.

For the put options, the knock-out price is quite low, about 1.8439, reflecting the small likelihood that the option will remain in the money without being knocked out at the barrier. Conversely, the put knock-in option has a higher value, approximately 2.9662, since it only becomes valuable if the barrier is breached, which is more likely when the underlying price declines, making the put more likely to finish in-the-money.

Option	**Calculated Price**
Call Knock-Out	9.6453
Call Knock-In	0.0415
Put Knock-Out	1.8439
Put Knock-In	2.9662

Table 1: Barrier Option prices using Boyle and Tian pricing method

4.4 Conclusion

We could observe that for minimal computation time, the finite difference methods allow to have a very good approximation of price, around 0.07% difference with regard to Black-Scholes price for Tian & Boyle, 0.45% for Crank-Nicolson, better than for Monte-Carlo simulation after one million simulations (4,15% difference) that required up to 4 5 seconds of calculation against less than 5 for FDM.

5. Greeks

For hedging and risk management purposes it is crucial to understand how option values change with marginal changes in a model parameter. Thanks to Greeks we can measure these sensitivities. While closed formulas for vanilla options are well established it is not the case for barrier options. However, using numeric procedures, we can study the behavior of delta, gamma, vega, rho and theta.

5.1 Delta

The delta of an option is the first partial derivative of the option's value and measures the rate of change of the option price relative to a change in the underlying asset price. It means that when the underlying asset price changes by one unit, the option's price will change by Δ units. Delta is given by :

$$\Delta = \frac{\partial f}{\partial S}$$

Where f is the option's price and S is the underlying asset price. Delta is used to determine the quantity of the underlying asset the portfolio should hold to mitigate risk. However, the portfolio will only be riskless for an infinitesimal period of time. Vanilla call options have a delta ranging between 0 to 1 and Vanilla put options have a delta ranging between 0 to -1 while the delta of a barrier options behave in a different way.

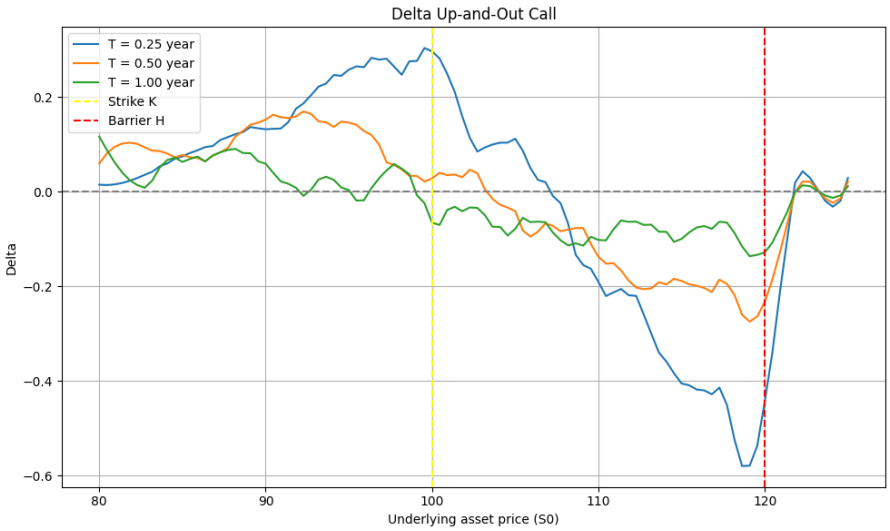


Figure 1: Delta of an Up-and-Out Call

We notice the delta of an up-and-out call can be either positive or negative and that its value grows as the underlying asset price approaches the barrier and maturity gets closer. Another

important characteristic of knock-out barrier options is that as soon as the barrier is touched, the delta drops to 0.

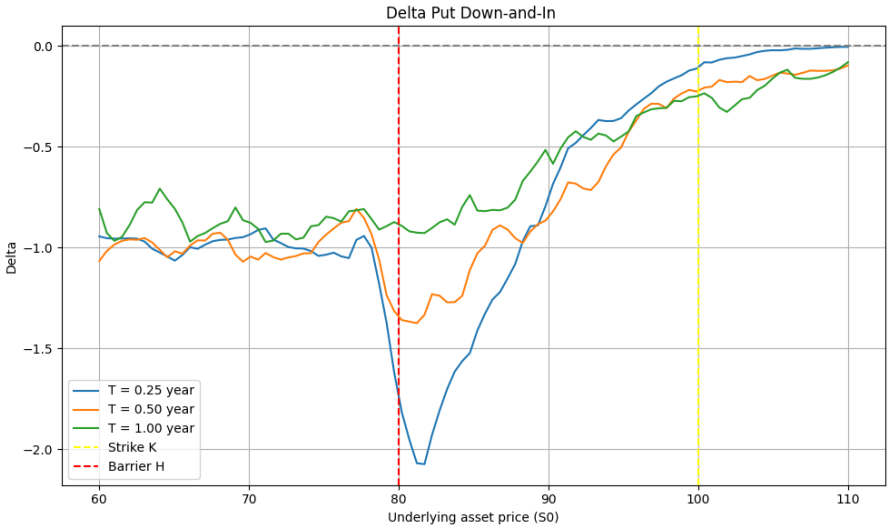


Figure 2: Delta of a Down-and-In Put

In figure 2 we observe that the delta of a barrier option can be larger than one in absolute value. This implies significant risks for trading activities as trading costs rise and the market may not offer adequate liquidity for rebalancing one's portfolio.

5.2 Gamma

The gamma measures the rate of change of the delta for a given change in the underlying asset price. It is given by :

$$\Gamma = \frac{\partial^2 f}{\partial^2 S}$$

Gamma plays an important role in mitigating risks. As Gamma gets smaller, delta changes more slowly which implies less rebalancing and thus reduced costs for the trader. On the contrary if Gamma takes large values, then delta changes rapidly along with the frequency of the portfolio adjustments that have to be made thus suggesting more costs. Furthermore, if gamma is negative, a portfolio readjustment signifies a loss whereas when gamma is positive a portfolio readjustment results in a profit.

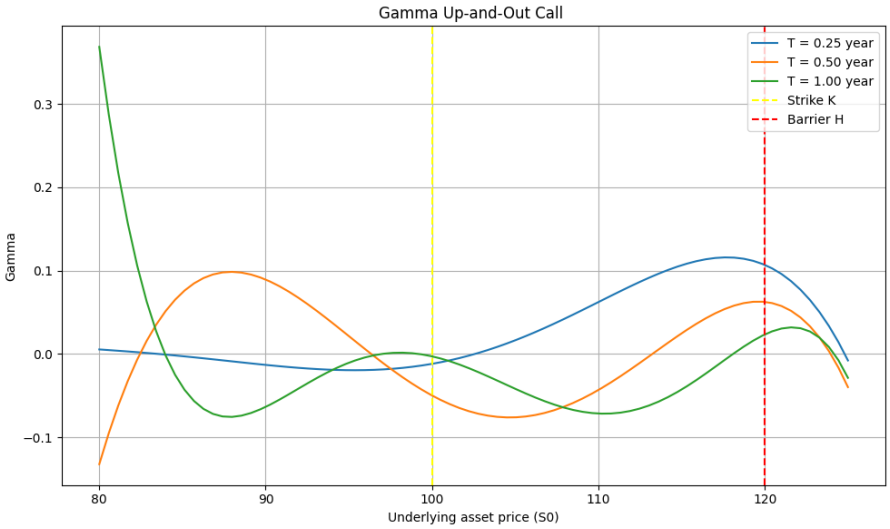


Figure 3: Gamma of an Up-and-Out Call

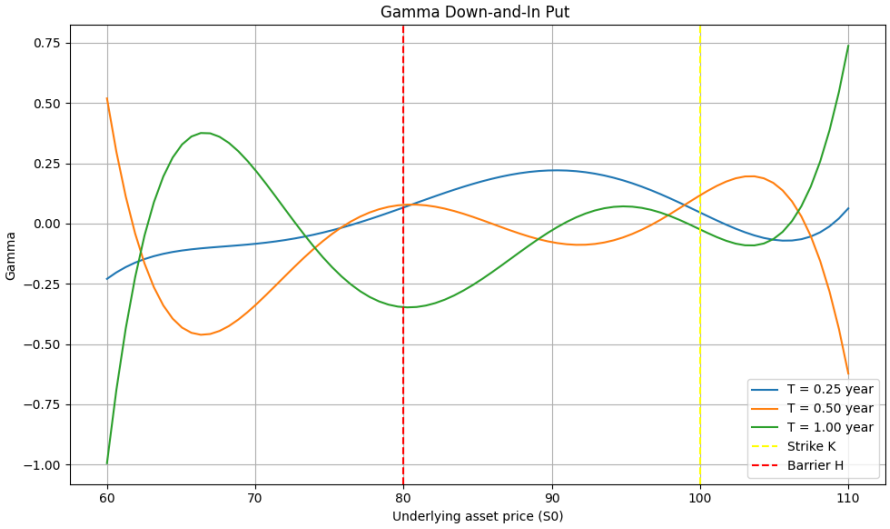


Figure 4: Gamma of a Down-and-In Put

We note that for barrier options, gamma can change sign contrary to vanilla options. Another important aspect is that gamma of barrier options can take greater values than vanilla options, especially near the barrier which means increased difficulties to remain delta neutral.

5.3 Theta

Theta measures the rate of change of an option price as times goes by. It is given by :

$$\Theta = \frac{\partial f}{\partial t}$$

While the Theta of a vanilla option is negative, the theta of a barrier option can sometimes be positive. Indeed, if the option is in money, as time to maturity decreases, the probability of the option being knocked out decreases as well. Thus, the barrier option gains value.

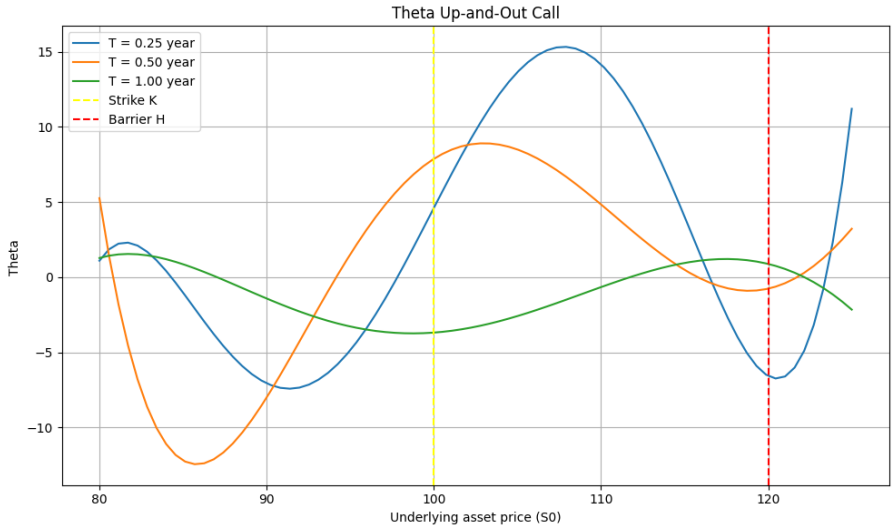


Figure 5: Theta of an Up-and-Out Call

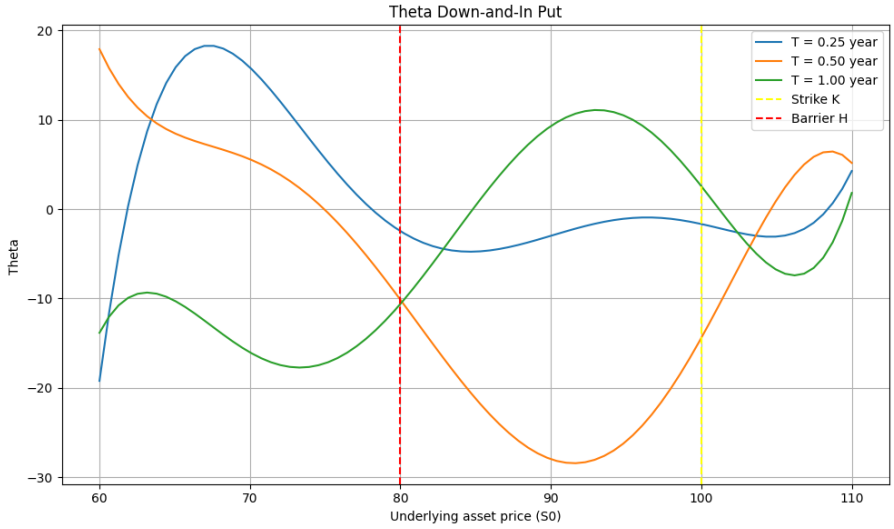


Figure 6: Theta of a Down-and-In Put

5.4 Vega

Vega measures the rate of change of the option price to a change in the underlying asset's volatility. It is given by

$$V = \partial f / \partial \sigma$$

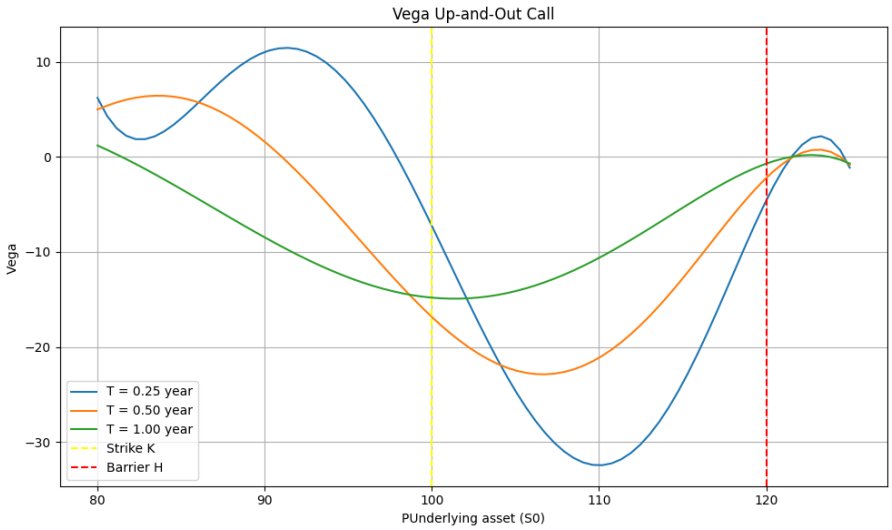


Figure 7: Vega of an Up-and-Out Call

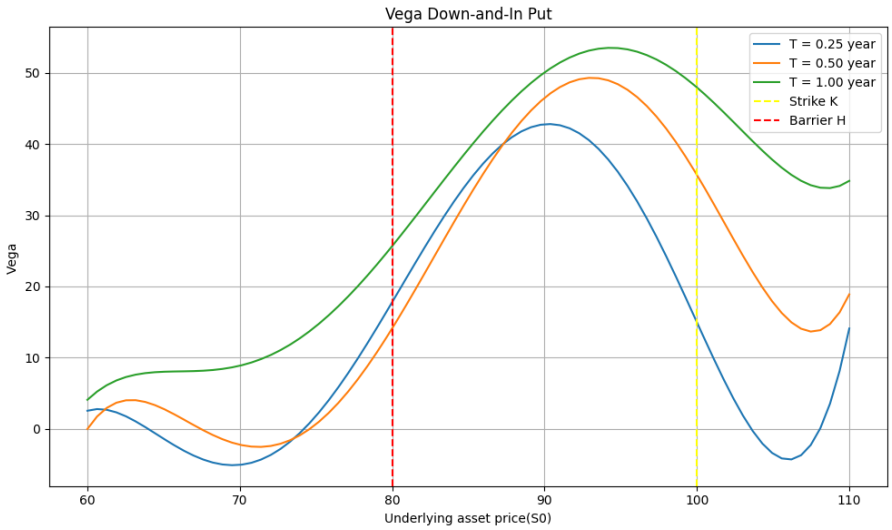


Figure 8: Vega of a Down-and-In Put

Knock-in options will rise in value as volatility rises because the probability of the vanilla option existing rises while Knock-out options will decrease in value as volatility rises because of the increased probability of the option expiring worthless.

5.5 Rho

Rho measures the rate of change of the option price to a change in interest rates and is given by

$$P = \frac{\partial f}{\partial r}$$

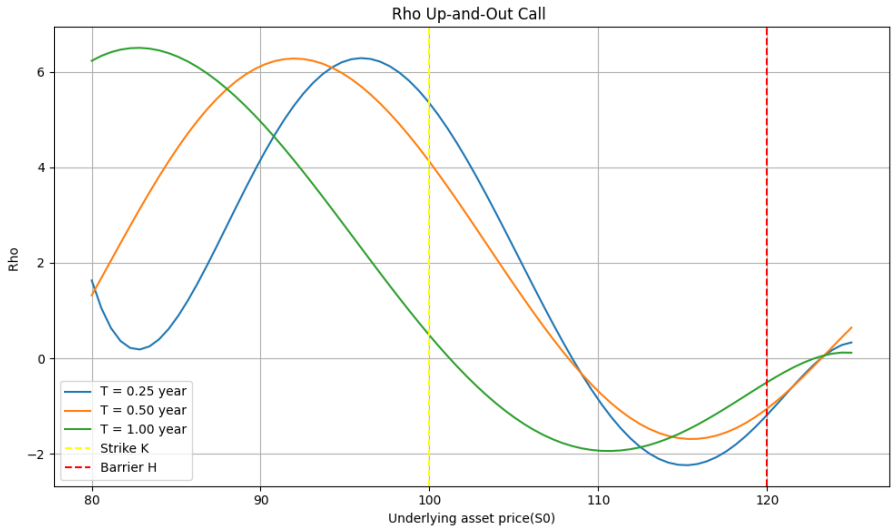


Figure 9: Rho of an Up-and-Out Call

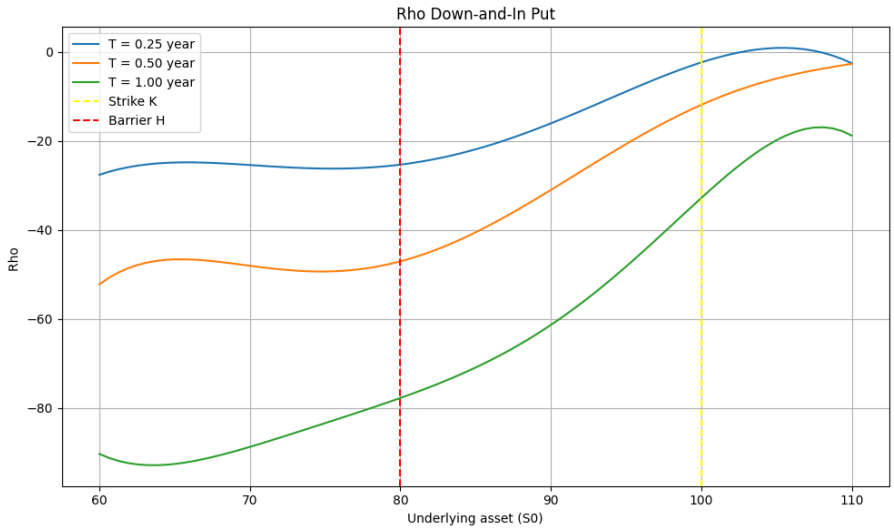


Figure 10: Rho of a Down-and-In Put

6. Hedging

6.1 Delta hedging

Using the Black Scholes Merton model, we set up a portfolio composed of a position in the derivative and a position in the derivative's underlying asset such that for an infinitesimal period of time the portfolio built is riskless and thus must earn the risk-free rate. This is possible because the source of uncertainty of the derivative is the same as the source of uncertainty of the underlying asset: the underlying asset price variation. For the portfolio to remain riskless, the quantity of the underlying asset in the portfolio must be adjusted as its price, and thus delta, changes over time. This type of dynamic hedging is referred to as delta hedging.

Up-and-Out call parameters:

Barrier H: 120

Strike K: 100

Asset Price S0: 100

Maturity: 1 month

Risk free rate r: 5%

Dividend q: 3%

Volatility sigma: 20%

Contract size: 100 shares

Hedging procedure:

Each day:

Compute the new delta of the option based on time left to maturity and the underlying asset price.

Random path followed by the underlying asset is

$$S_{t+1=}\, S_t.\, exp\, (\left(r - \frac{\sigma^2}{2}\right)\Delta t + \, \sigma\sqrt{\Delta t} Z_t$$

Where:

$$Z_t \sim N(0,1)$$

$$\Delta t = \frac{1}{252}$$

- Adjust the position in the underlying to match the new delta by buying or selling shares of the underlying.

Delta of call in the Black Scholes model

$$\Delta_{call} = exp(-qt) \,.\, \phi(d1)$$

Where:

$$d1 = (ln\left(\frac{S_0}{K}\right) + \left(r - q + \frac{\sigma^2}{2}\right)T\,)\frac{1}{\sigma\sqrt{T}}$$

To maintain a delta-neutral portfolio, we must adjust the number of units of the underlying asset held so that the portfolio delta equals the new delta of the option. Let

- Δ_{target} be the new delta of the option at time t
- $\Delta_{current}$ current number of underlying shares held in the hedge portfolio

The required adjustment in the underlying position is:

$$Adjustment = \Delta_{target} - \Delta_{current}$$

If the Adjustment > 0: buy shares of the underlying

If the Adjustment < 0: sell shares of the underlying

This adjustment ensures that small movements in the underlying price SS have minimal impact on the value of the portfolio because first-order risk is neutralized.

When buying or selling shares, there is a cash flow associated with the transaction. It modifies the cash account in the hedgign portfolio. We assume continuous trading and no transaction costs for simplicity, as in the Black-Scholes framework.

$$Cash\ Flow_t = -Adjustment \,.\, S_t$$

Newt, we account for the interest earned or paid on the cash position, as well as any dividend income or expense associated with holding the underlying asset. After adjusting the hedge by buying or selling the underlying, the remaining funds are either invested at the risk-free rate (if positive) or borrowed (if negative), accruing or incurring interest accordingly. The interest on the cash position is computed as the cash balance multiplied by the risk-free rate and the

time increment Δt. Additionally, if the underlying asset pays continuous dividends at a yield q, the portfolio receives or pays an amount proportional to the number of shares held, the asset price, and the dividend yield over the period. The total interest component for the period thus includes both the cash interest and the dividend effect.

$$Interest = Cash\ .r.\Delta t\ - asset\ owned\ .S_t.q.\Delta t$$

The total hedging cost is updated by accumulating the net cash flows associated with maintaining the hedge. This includes the cash spent or received during the asset adjustment and the interest and dividend terms. The updated cost reflects the input required to maintain the delta-neutral position over time. Tracking this cumulative cost allows us to measure the accuracy of the hedging strategy, especially when compared to the theoretical option price. In an ideal frictionless market under the Black-Scholes assumptions, the total hedging cost should converge to the fair value of the option at inception.

$$Cost_t = Cost_{t-1} + Cash\ Flow_t + Interest_t$$

Day	Asset Price	Delta	Position	Cashflow	Interest & Yield	Total Cost
0	100	1,710300962	171	-17100	0	-17100
1	100,4434887	1,729969435	173	-200,8869773	-2,718799795	-17303,6058
2	100,3198429	1,726367403	173	0	-2,749683083	-17306,3555
3	100,9003685	1,753106212	175	-201,8007369	-2,7559339	-17510,9121
4	102,2787376	1,811844664	181	-613,6724255	-2,80259718	-18127,3872
5	102,0656061	1,807897368	181	0	-2,897987299	-18130,2851
6	101,8529336	1,802972803	180	101,8529336	-2,895983505	-18031,3282
7	103,296004	1,86462612	186	-619,7760238	-2,895565141	-18653,9998
8	104,0046431	1,894568188	189	-312,0139294	-3,002075194	-18969,0158
9	103,5705614	1,887315152	189	0	-3,04701562	-18972,0628
10	104,0723825	1,911787476	191	-208,1447649	-3,05296339	-19183,2605
11	103,6436102	1,903919128	190	103,6436102	-3,081430383	-19082,6983
12	103,2144785	1,893076051	189	103,2144785	-3,060431439	-18982,5443
13	103,4372047	1,909680026	191	-206,8744093	-3,046857471	-19192,4656

14	101,6890731	1,824315558	182	915,2016582	-3,060122356	-18280,324
15	100,1383836	1,72572152	173	901,2454527	-2,898356699	-17381,9769
16	99,63801916	1,689552746	169	398,5520767	-2,750436838	-16986,1753
17	98,74302748	1,620607643	162	691,2011924	-2,678444274	-16297,6525
18	99,01984962	1,637296875	164	-198,0396992	-2,571664874	-16498,2639
19	98,2220773	1,568845231	157	687,5545411	-2,595567095	-15813,3049
20	96,99400849	1,469288543	147	969,9400849	-2,475212355	-14845,8401
21	98,26876702	1,555163049	156	-884,4189032	-2,332653305	-15732,5916
22	98,07131032	1,526858664	153	294,2139309	-2,471435146	-15440,8491
23	98,13032669	1,518656334	152	98,13032669	-2,425517174	-15345,1443
24	96,89266251	1,398081858	140	1162,71195	-2,398983645	-14184,8314
25	96,42389525	1,346733101	135	482,1194763	-2,210757793	-13704,9226
26	96,51922622	1,326313189	133	193,0384524	-2,135216265	-13514,0194
27	95,53458783	1,274786409	127	573,207527	-2,096992014	-12942,9089
28	95,85487627	1,260773681	126	95,85487627	-2,008635956	-12849,0626
29	95,34333483	1,258540243	126	0	-1,989783605	-12851,0524
30	95,09589589	0	0	11982,08288	-1,988125212	-870,957647

Table 1: Simulation of the delta hedging procedure

In this first example the asset price fluctuates between 95 and 104 and has never breached the knock-out barrier set at 120. Because the option is never knocked out, we can hedge like we would have done for a vanilla call option. Initially, delta is 1.71 which means the option is deep in the money and thus very sensitive to a movement in the underlying asset price. Delta decreases as expiration approaches, reaching 0 at maturity. Each day we adjust the quantity owned in the underlying to match the new delta. These adjustments are aligned with cash flows when buying or selling shares in the underlying.

Initially total cost amount to 17 100€, it changes during the option lifetime and ends up to 870.96€. It represents the hedging cost, an approximation of the option price according to the simulated path of the underlying asset price.

Day	Asset Price	Delta	Position	Cashflow	Interest & Yield	Total Cost	Knocked Out
0	110	77,85896	78	-8580	0	-8580	FALSE
1	107,019	71,81511	72	642,1137	-2,69613	-7940,58	FALSE
2	109,7294	77,96102	78	-658,377	-2,51552	-8601,47	FALSE
3	110,5015	79,82396	80	-221,003	-2,73169	-8825,21	FALSE
4	106,3733	71,04601	71	957,36	-2,76254	-7870,61	FALSE
5	104,8213	67,25334	67	419,285	-2,44549	-7453,77	FALSE
6	109,2621	78,437	78	-1201,88	-2,34781	-8658	FALSE
7	102,769	61,6813	62	1644,305	-2,66907	-7016,37	FALSE
8	101,6522	58,26224	58	406,6089	-2,13882	-6611,9	FALSE
9	104,9346	68,59705	69	-1154,28	-2,0324	-7768,21	FALSE
10	102,6554	61,75839	62	718,5881	-2,38012	-7052	FALSE
11	100,9023	55,85226	56	605,4139	-2,13906	-6448,73	FALSE
12	100,6498	54,95108	55	100,6498	-1,94518	-6350,02	FALSE
13	104,4917	68,57816	69	-1462,88	-1,93838	-7814,85	FALSE
14	102,8108	63,11217	63	616,8645	-2,38898	-7200,37	FALSE
15	101,6549	58,95821	59	406,6195	-2,18448	-6795,93	FALSE
16	100,5364	54,51329	55	402,1456	-2,04754	-6395,84	FALSE
17	106,2702	76,49483	76	-2231,67	-1,95742	-8629,47	FALSE
18	112,2769	91,50805	92	-1796,43	-2,72023	-10428,6	FALSE
19	115,14	95,77417	96	-460,56	-3,32189	-10892,5	FALSE
20	116,252	97,2633	97	-116,252	-3,48081	-11012,2	FALSE
21	118,4181	98,77431	99	-236,836	-3,54272	-11252,6	FALSE
22	122,9362	0	0	12170,68	-3,67116	914,3975	TRUE
23	120,0569	0	0	0	0,192551	914,5901	TRUE
24	123,6524	0	0	0	0,192551	914,7826	TRUE
25	119,7924	0	0	0	0,192551	914,9752	TRUE
26	117,8688	0	0	0	0,192551	915,1677	TRUE
27	120,5795	0	0	0	0,192551	915,3603	TRUE
28	116,3021	0	0	0	0,192551	915,5528	TRUE
29	115,8786	0	0	0	0,192551	915,7454	TRUE
30	113,376	0	0	0	0,192551	915,9379	TRUE

Table 2: Simulation of the delta hedging procedure with option knocking out

In this second scenario the initial underlying asset price is 110 and the delta is approximately 0.7786. At inception the hedger buys 78 units of the underlying which costs him 110 × 78 = 8580€. The hedger buy these units by borrowing the corresponding amount at the risk free rate. On day 1, the asset decreases to 107.02€ and delta falls to 0.7181. Therefore the hedger reduces his position to 72 shares by selling 6 shares at 107.02€. He recovers 642.11€ and pays interests. The total costs amounts then to -7940.58€. As the underlying asset price fluctuates in the following days, delta is adjusted daily by buying or selling units, and interest and dividend yield are accrued. The hedger keep on with the same procedure until day 22, when the underlying asset price breaches the knock out barrier set at 120. Consequently, delta falls to 0 and the hedger sells its whole position in the underlying shares at market price. He receives 99 × 122.94 = 12170.68€, bringing total costs to a net gain of 915.94€ after interests income. This hedging strategy, while costly during the option's life results in a profit due to the knock out event triggering the liquidation of the hedged position at a favorable price.

If the hedger rebalanced his portfolio continuously and followed the Black Scholes Merton model which assumes constant volatility, he would always be able to match his hedging costs to the initial option price. Thus resulting in a net P&L of zero.

However in practise it is not possible to rebalance a portfolio continuously because markets close, there are transaction costs, liquidity issues…The hedger would have to rebalance his portfolio at discrete times which implies P&L.

Furthermore Delta hedging shows weaknesses around the barrier because it can not rapidly adjust to a price jump.

6.2 Boundary Replication:

Boundary replication is a hedging technique for barrier options. It allows to better assess the price jump at the barrier by replicating the payoff discontinuity using vanillas options such as calls, puts or forwards. Contrary to delta hedging, that dynamically adjust the hedger's position in the underlying asset according to its price varirations, boundary replication focus on building a semi-static replication of the payoff at the barrier by selecting a combinations of vanillas with well chosen strikes, especially those around the barrier. It is based upon the brownian motion symmetry.

Theorem(The reflection principle)

Given a standard Brownian motion $B(t)$, for every $a \geq 0$

$$P(M(t) \geq a) = 2P(B(t) \geq a) = 2\,\frac{1}{\sqrt{2\pi t}}\int_a^{\infty} e^{\frac{-x^2}{2t}}\,dx$$

Proof: We have

$$P(B(t) \geq a) = P(B(t) \geq a, M(t) \geq a) + P(B(t) \geq a, M(t) < a)$$

But $P(B(t) \geq a, M(t) < a) = 0$ since $B(t) \leq M(t)$. Let $T_a = inf\{s \geq 0, B_s = a\}$ be the first hitting time of level $a > 0$,

$P(B(t) \geq a) = P(B(t) \geq a | M(t) \geq a) P(M(t) \geq a)$

$= P(B(t) \geq a | T_a \leq t) P(M(t) \geq a)$

We have that $B(T_a + s) - a$ is a Brownian motion conditionned to $T_a \leq t$,

$$P(B(t) \geq a \mid T_a \leq t) = P(B(T_a + (t - T_a)) - a \geq 0 \mid T_a \leq t) =)\frac{1}{2}$$

Since the Brownian motion satisfies $P(B(t) \geq 0) = \frac{1}{2}$ for every t. Applying the identity we obtain,

$$P(B(t) \geq a) = \frac{1}{2} P(M(t) \geq a)$$

The reflection principle is fundamental in hedging barrier options as it allows us to calculate the probability of breaching a barrier.

By setting up a portfolio with vanilla options that has the same value everywhere in the boundary and the same cashflows within the boundary, the model guarantees that the replication portfolio and the target option will have the same value everywhere at the boundary.

The major advantage of the boundary replication hedging method is its lower need for rebalancing which results in lower costs and makes it less sensitive to time discretization errors. It is also more robust price jumps and market shocks as the hedge is embedded within the derivative instrument used.

We can use the method to replicate barrier options. Consider the following up-and-out call option:

Asset price – 100

Exercise price – 100

Barrier – 120

Time to expiration – 1 year

Risk-free interest rate – 5%

Asset yield – 3%

Volatility – 20%

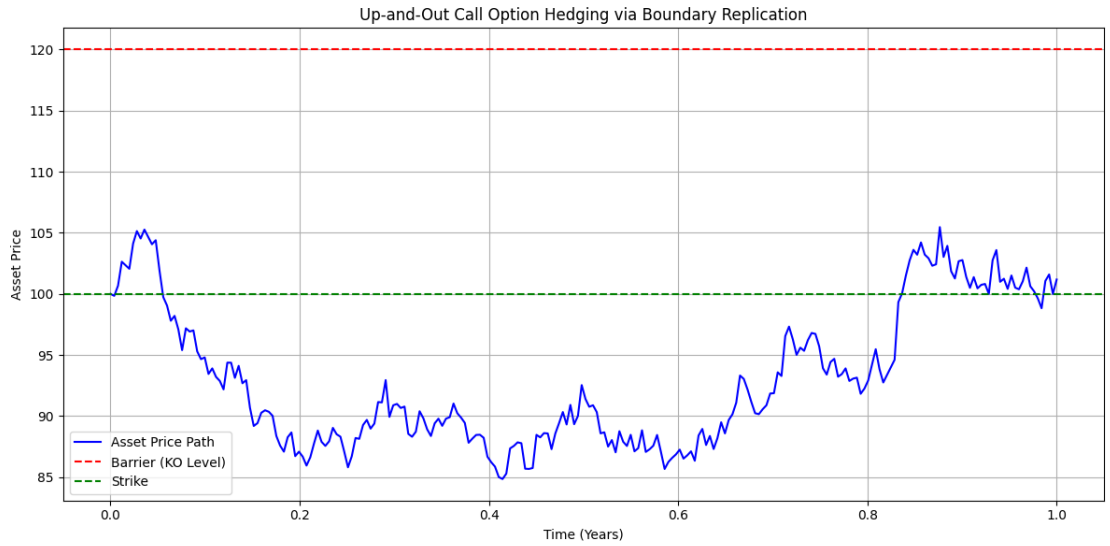


Figure 11: Path of the Up-and-Out call underlying asset price with Strike at level 100 and barrier at level 120

We simulated the path of its underlying asset price over one year. By implementing the boundary replication strategy we obtain a replication portfolio composed of:

- One long position in a vanilla European call with strike 100
- One short position in a vanilla European call with strike at the barrier level 120

This combination allows to replicate the payoff of an up-and-out call. As the payoff of an Up-and-Out call is the same as a vanilla call option while the knock out barrier is not breached, the long position in the call provides the basic call payoff, and the short call cancels it if the asset price crosses the barrier.

In this scenario we focus on replicating the up-and-out call payoff at expiration without acknowledging intermediate conditions. This technique is sufficient if everything goes well but if during the life of the option conditions change and barrier is breached the whole method collapse.

We will try now to take into account intermediate conditions by building the structure step by step. This will allow us to first reproduce the payoff at maturity but also to ensure that the portfolio has a zero value as soon as the barrier is reached, at any time (or at least at several key moments, in this case every 6 months).

At maturity, if the underlying asset price $S < H$ (the barrier), the up-and-out call option behaves like a vanilla call with a payoff equal to $max\ (S_T - K, 0)$. We begin by buying a vanilla call option with strike 100.

However, a vanilla call retains value even if the underlying price exceeds the barrier, while the up-and-out call becomes worthless. To correct for this, we need to subtract the value of another option that cancels this residual value above the barrier.

For $S = 120$:

- The value of a call $(K = 100, T = 1)$is 21.29
- The value of a call $(K = 120, T = 1)$ is 7.24

We sell a quantity q of the second call such that:

$$21.29 - 7.24.\, q = 0 \qquad => \qquad q = 2.9423521$$

The replication portfolio becomes:

Option	Quantity	Strike	Expiration	Value(S=100)
Call	1.000000	100	1	8.652529
Call	-2.942353	120	1	-7.272475
			Total	1.38005

Table 3: Replication Portfolio of an Up-and-Call

Notice that at inception the replication portfolio value is 1.38005, much lower than the theoretical value of the up-and-out call option which is 8.65253. There is a significant replication error as it can be observed in Figure 12 which plots the difference between the payoff of a barrier option and the payoff of its replicating portfolio across different underlying asset levels or time horizons.

We can minimize the replication error by rebalancing our portfolio at the barrier more often. Table 4 shows the replication portfolio when the boundary condition is modelled every month. The replication error is minimized as can be seen in Figure 13.

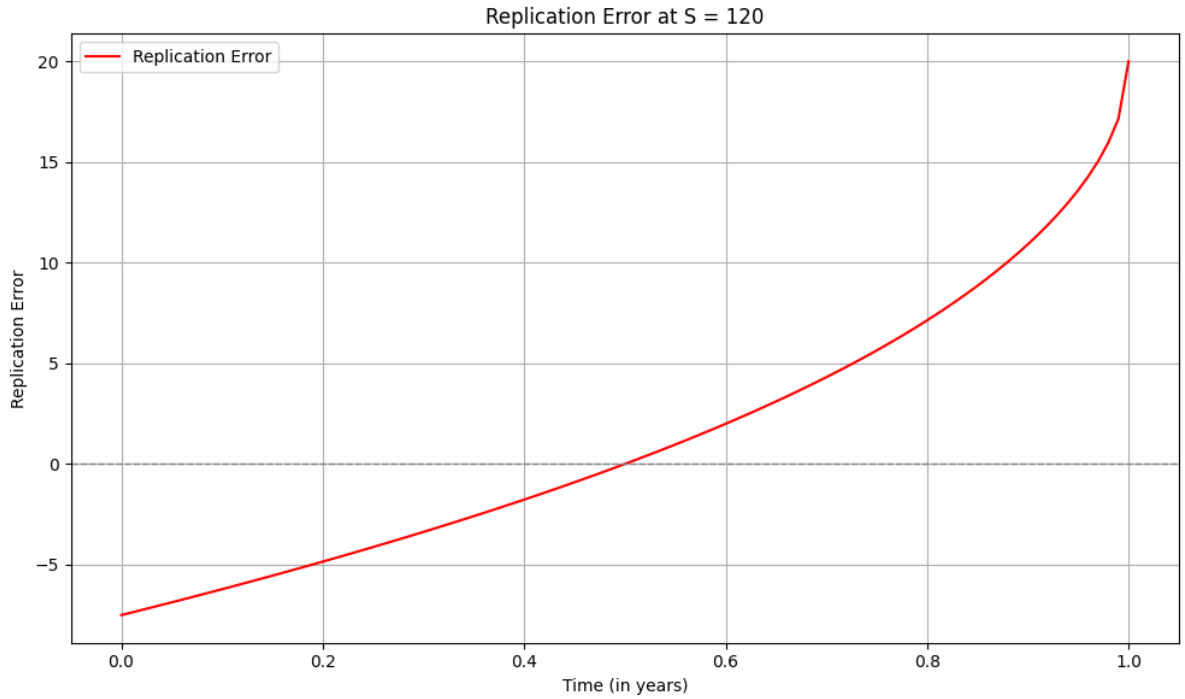


Figure 12: Up-and-out call replication error, with matching every 6 months.

Option	Quantity	Strike	Expiration	Value(S=100)
Call	0.051305	120	0.17	0.002276
Call	0.062488	120	0.25	0.010337
Call	0.077971	120	0.33	0.027137
Call	0.100303	120	0.42	0.057255
Call	0.134252	120	0.50	0.109914
Call	0.189534	120	0.58	0.205084
Call	0.288434	120	0.67	0.390693
Call	0.491204	120	0.75	0.801678
Call	1.006955	120	0.83	1.925209
Call	2.979403	120	0.92	6.531451
Call	1.000000	100	1.00	8.652529
Call	-7.045910	120	1.00	-17.415047
			Total	1.298514

Table 4: Replication Portfolio of an Up-and-Call

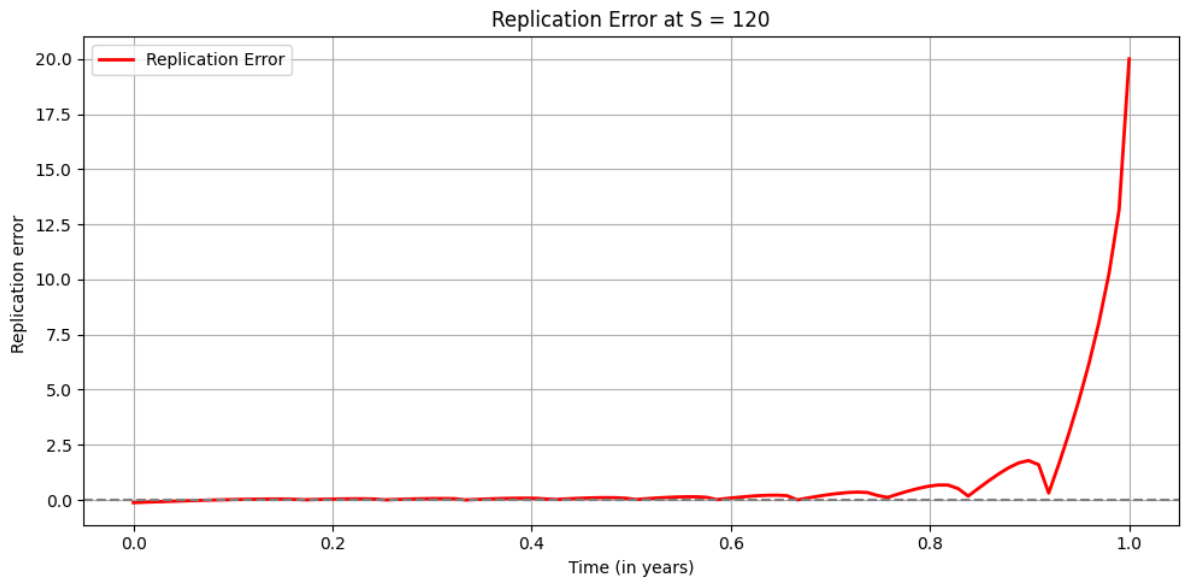


Figure 13: Up-and-out call replication error, with matching every month.

A significant drawback is that it relies on precise knowledge of market parameters (implied volatility, correlations, etc.), and it may not adapt well if conditions change abruptly. Additionally, some of the options required for the replication may not exist in the market or may be illiquid, making the theoretical strategy difficult to implement in practice.

In summary, boundary replication offers a more stable and less management-intensive alternative to delta hedging, but at the cost of reduced flexibility and increased dependence on the accuracy of initial assumptions.

Static hedging techniques are proposed to enable less hedging error for barrier options. While these methods show promising results in the geometric Brownian motion assumptions where they were developed we show that they are prone to hedging errors in real market condition where volatility is stochastic.

Boundary replication techniques prove their theoretical value but run into problems when the volatility surface changes and leads to situations where the replication portfolio value at the barrier is not what was expected.

Bibliography:

Nneka Umeorah & Phillip Mashele | (2019) A Crank-Nicolson finite difference approach on the numerical estimation of rebate barrier option prices, Cogent Economics & Finance, 7:1, 1598835

Tian (Sam) , Phelim P. Boyle Yisong (1998) 'An explicit finite difference approach to the pricing of barrier options', Applied Mathematical Finance, 5:1, 17 – 43

Antonin Chaix, Volatilité locale (Dupire) : PDE

Antonin Chaix, Volatilité locale (Dupire) : Calibration et Monte Carlo

O.L Babasola, Philip Ngare, E.A Owoloko (2018), Crank Nicolson Approach for the Valuation of the Barrier Options, International Journal of Applied Mathematical Science, 11 :1, 7-21

Simon Fraser University, « Numerically Solving PDE's : Crank-Nicholson Algorithm », 8-14

Mohammed Bouzoubaa, Adel Osseiran (2010), Exotic Options & Hybrids – A guide to structuring, pricing and trading, Whiley

Static Options Replication, May 1994 Emanuel Derman Deniz Ergener Iraj Kani, Goldman Sachs Quantitative Strategies Research Notes

Derman, E and I. Kani (1996), The Ins and Outs of Barrier Options: Part 1, Derivatives Quarterly Winter 1996, 55-67

Hull, J. C. (2008), Options, Futures and other Derivatives, 7th Edition, Prentice Hall

M. Villaverde, Hedging European and Barrier Options using Stochastic Optimization(2003), University of Cambridge Research Papers

.

Appendix

Monte-Carlo Simulation for pricing of Down-In Put Options – Flat Volatility

```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt


def EU_DnI_Put_MC(S0, K, r, sigma, T, nb_simu, H):
    dt = 1 / 252
    nb_step = int(T * 252)  # Nombre de pas
    Z = np.random.normal(0, 1, (nb_simu, nb_step))  # Bruit normal
    S = np.zeros((nb_simu, nb_step + 1))
    S[:, 0] = S0  # Initialisation du prix

    # Simulation des trajectoires
    for i in range(1, nb_step + 1):
        S[:, i] = S[:, i - 1] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[:, i - 1])

    # Calcul des payoffs
    payoff = np.zeros(nb_simu)
    for k in range(1, nb_simu):
        if np.min(S[k, :]) <= H:  # Vérifie si la barrière est franchie
            payoff[k] = np.maximum(0, K- S[k, -1])  # Payoff du put

    # Calcul du prix par Monte-Carlo
    return np.exp(-r * T) * np.mean(payoff)
```

```
print(EU_DnI_Put_MC(100, 100, 0.05, 0.17959625569569046, 1, 1000000, 80))
```

Monte-Carlo Simulation for pricing of Down-In Put Options – Local Volatility

```
import numpy as np
from scipy.interpolate import interp1d
from scipy.stats import norm
import time as timer
import sys

# Constantes du modèle
r = 0.05  # taux d'intérêt constant
S0 = 100
T = 1
K = 100
H = 80

# Paramètres SABR
sig0 = 0.17
alpha = 0.65
rho = -0.3
beta = 1

# Fonction pour afficher une barre de progression
def progress_bar(iteration, total, prefix='', suffix='', decimals=1, length=50, fill='█', print_end='\r'):
    percent = ("{0:." + str(decimals) + "f}").format(100 * (iteration / float(total)))
    filled_length = int(length * iteration // total)
```

```
    bar = fill * filled_length + '-' * (length - filled_length)
    sys.stdout.write(f'\r{prefix} |{bar}| {percent}% {suffix}')
    sys.stdout.flush()
    if iteration == total:
        print()

# SABR Volatility Function
def SABR(alpha, beta, rho, K, FwP, sig0, T):
    epsi = 1e-07
    logFK = np.log(FwP / K)
    FwPK = (FwP * K) ** (1 - beta)

    e1 = ((1 - beta) * sig0) ** 2 / (24 * FwPK)
    e2 = rho * beta * alpha * sig0 / (4 * np.sqrt(FwPK))
    e3 = (2 - 3 * rho ** 2) * alpha ** 2 / 24
    e4 = (1 - beta) ** 2 / 24 * logFK ** 2
    e5 = ((1 - beta) * logFK) ** 4 / 1920

    A = sig0 * (1 + (e1 + e2 + e3) * T) / (np.sqrt(FwPK) * (1 + e4 + e5))

    z = alpha / sig0 * np.sqrt(FwPK) * logFK
    x_factor = np.ones_like(z)
    mask = np.abs(z) > epsi
    x_factor[mask] = np.log((np.sqrt(1 - 2 * rho * z[mask] + z[mask] ** 2) + z[mask] - rho) /
(1 - rho))
    result = np.ones_like(z)
    result[mask] = z[mask] / x_factor[mask]
```

```
    return A * result


# Volatilité locale approximée via Dupire
def vol_loc(t, S):
    FwP = S0 * np.exp(r * t)
    t = np.where(t == 0, 1e-06, t)
    dt = 1e-07
    dS = S * 1e-07

    vol_init = SABR(alpha, beta, rho, S, FwP, sig0, t)
    vol_init = np.where(vol_init == 0, 1e-08, vol_init)

    dvol_dt = (SABR(alpha, beta, rho, S, S0 * np.exp(r * (t + dt)), sig0, t + dt) - vol_init) / dt
    dvol_dK = (SABR(alpha, beta, rho, S + dS, FwP, sig0, t) - vol_init) / dS
    d2vol_dK2 = (
        SABR(alpha, beta, rho, S + dS, FwP, sig0, t) +
        SABR(alpha, beta, rho, S - dS, FwP, sig0, t) -
        2 * vol_init
    ) / dS**2

    sqrt_t = np.sqrt(t)
    d1 = (np.log(S0 / S) + np.log(FwP / S0) + 0.5 * vol_init ** 2 * t) / (vol_init * sqrt_t)

    num = 2 * dvol_dt + vol_init / t + 2 * S * r * dvol_dK
    den = S ** 2 * (d2vol_dK2 - d1 * sqrt_t * dvol_dK ** 2 + (1 / (S * sqrt_t) + d1 * dvol_dK)
** 2 / vol_init)
```

```
    den = np.where(np.abs(den) < 1e-12, 1e-12, den)

    var = num / den
    var = np.where(var < 0, 0, var)
    return np.sqrt(var)

# Simulation Monte Carlo
nb_simu = 1000000
nb_steps = 252
dt = T / nb_steps
Z = np.random.normal(0, 1, (nb_simu, nb_steps))
S = np.zeros((nb_simu, nb_steps + 1))
S[:, 0] = S0

start_time = timer.time()
print(f"Simulation de {nb_simu} trajectoires avec {nb_steps} pas de temps...")

for i in range(1, nb_steps + 1):
    time = i * dt
    current_price = S0 * np.exp(r * time)

    vol = SABR(alpha, beta, rho, current_price, S0 * np.exp(r * T), sig0, time)
    Smax = current_price * np.exp(-0.5 * vol ** 2 * time + 5 * vol * np.sqrt(time))
    Smin = current_price * np.exp(-0.5 * vol ** 2 * time - 5 * vol * np.sqrt(time))
    S_likely_range = np.linspace(Smin, Smax, 100)
```

```
    vol_range = vol_loc(time - dt, S_likely_range)
    local_vol_surf = interp1d(S_likely_range, vol_range, kind='cubic', fill_value='extrapolate')

    current_sigma = local_vol_surf(S[:, i - 1])
    S[:, i] = S[:, i - 1] * np.exp((r - 0.5 * current_sigma ** 2) * dt + current_sigma * np.sqrt(dt)
* Z[:, i - 1])

    elapsed_time = timer.time() - start_time
    estimated_total_time = elapsed_time * nb_steps / i
    remaining_time = max(0, estimated_total_time - elapsed_time)
    mins, secs = divmod(int(remaining_time), 60)
    progress_bar(i, nb_steps, prefix='Progrès:', suffix=f'Temps restant ~
{mins:02d}:{secs:02d}', length=40)

# Payoff d'un Put Down-&-In
payoff = np.zeros(nb_simu)
for k in range(nb_simu):
    if np.min(S[k, :]) <= H:
        payoff[k] = np.maximum(0, K - S[k, -1])

D_I_put_price = np.exp(-r * T) * np.mean(payoff)
print(f"Le prix du put Down-&-In en volatilité locale est : {D_I_put_price:.6f}")

def black_scholes_down_in_put(S0, K, H, r, T, sigma):
    q = 0.0  # dividend yield assumed to be 0
    lambda_ = (r - q + 0.5 * sigma ** 2) / sigma ** 2
```

```
    x1 = np.log(S0 / H) / (sigma * np.sqrt(T)) + lambda_ * sigma * np.sqrt(T)
    y1 = np.log(H /S0) / (sigma * np.sqrt(T)) + lambda_ * sigma * np.sqrt(T)
    y  = np.log(H ** 2 / (S0 * K)) / (sigma * np.sqrt(T)) + lambda_ * sigma * np.sqrt(T)

    term1 = -S0 * np.exp(-q * T) * norm.cdf(-x1)
    term2 = K * np.exp(-r * T) * norm.cdf(-x1 + sigma * np.sqrt(T))
    term3 = S0 * np.exp(-q * T) * ((H / S0) ** (2 * lambda_)) * (norm.cdf(y) - norm.cdf(y1))
    term4 = -K * np.exp(-r * T) * ((H / S0) ** (2 * lambda_ - 2)) * (norm.cdf(y - sigma *
np.sqrt(T)) - norm.cdf(y1 - sigma * np.sqrt(T)))

    return term1 + term2 + term3 + term4

sigma_bs = SABR(alpha, beta, rho, K, S0 * np.exp(r * T), sig0, T)
bs_down_in_price = black_scholes_down_in_put(S0, K, H, r, T, sigma_bs)
print(f"Le prix théorique Black-Scholes (vol SABR ATM) est : {bs_down_in_price:.6f}")
print("volatilité implicite : ",sigma_bs)
```

Implicit Finite Difference Method with Crank-Nicolson

```
import numpy as np
from scipy.linalg import solve_banded

def crank_nicolson_vanilla_put(S0, K, r, sigma, T, Smax, M, N):
    dt = T / N
    dS = Smax / M
    S = np.linspace(0, Smax, M+1)
    V = np.maximum(K - S, 0)
```

```
j = np.arange(1, M)
alpha = 0.25 * dt * (sigma**2 * j**2 - r * j)
beta  = -0.5 * dt * (sigma**2 * j**2 + r)
gamma = 0.25 * dt * (sigma**2 * j**2 + r * j)

ab_A = np.zeros((3, M-1))
ab_A[0, 1:] = -gamma[:-1]
ab_A[1, :]  = 1 - beta
ab_A[2, :-1] = -alpha[1:]

A_up = gamma
A_mid = 1 + beta
A_low = alpha

for n in range(N):
    rhs = (
        A_low * V[0:M-1] +
        A_mid * V[1:M] +
        A_up * V[2:M+1]
    )
    rhs[0] += alpha[0] * K * np.exp(-r * dt * (n + 1))
    V[1:M] = solve_banded((1, 1), ab_A, rhs)
    V[0] = K * np.exp(-r * dt * (n + 1))
    V[M] = 0
```

```
    return np.interp(S0, S, V)


def crank_nicolson_put_down_out(S0, K, r, sigma, T, B, Smax, M, N, rebate=0.0):
    dt = T / N
    dS = Smax / M
    S = np.linspace(0, Smax, M+1)
    V = np.maximum(K - S, 0)

    j = np.arange(1, M)
    alpha = 0.25 * dt * (sigma**2 * j**2 - r * j)
    beta  = -0.5 * dt * (sigma**2 * j**2 + r)
    gamma = 0.25 * dt * (sigma**2 * j**2 + r * j)

    ab_A = np.zeros((3, M-1))
    ab_A[0, 1:] = -gamma[:-1]
    ab_A[1, :]  = 1 - beta
    ab_A[2, :-1] = -alpha[1:]

    A_up = gamma
    A_mid = 1 + beta
    A_low = alpha

    for n in range(N):
       rhs = (
          A_low * V[0:M-1] +
          A_mid * V[1:M] +
```

```
            A_up * V[2:M+1]
        )
        rhs[0] += alpha[0] * K * np.exp(-r * dt * (n + 1))
        V[1:M] = solve_banded((1, 1), ab_A, rhs)
        V[S < B] = rebate
        V[0] = K * np.exp(-r * dt * (n + 1))
        V[M] = 0

    return np.interp(S0, S, V)

def crank_nicolson_put_knock_in(S0, K, r, sigma, T, B, Smax, M=5000, N=5000,
rebate=0.0):
    # Base result with M and N
    V1 = crank_nicolson_vanilla_put(S0, K, r, sigma, T, Smax, M, N)
    D1 = crank_nicolson_put_down_out(S0, K, r, sigma, T, B, Smax, M, N, rebate)

    # Finer grid result with 2M and 2N
    V2 = crank_nicolson_vanilla_put(S0, K, r, sigma, T, Smax, 2*M, 2*N)
    D2 = crank_nicolson_put_down_out(S0, K, r, sigma, T, B, Smax, 2*M, 2*N, rebate)

    # Richardson extrapolation (p=2)
    knock_in_1 = V1 - D1
    knock_in_2 = V2 - D2
    p = 2
    return ((2**p) * knock_in_2 - knock_in_1) / (2**p -1)

if __name__ == "__main__":
```

```
    S0 = 100
    K = 100
    r = 0.05
    sigma = 0.179596255695
    T = 1.0
    B = 80
    Smax = 150
    price = crank_nicolson_put_knock_in(S0, K, r, sigma, T, B, Smax)
    print(f"Prix put down-in (Crank-Nicolson, Richardson) : {price:.4f}")
```

Tian and Boyle

```
import numpy as np
from dataclasses import dataclass

@dataclass
class BarrierOption:
    S0: float
    X: float
    r: float
    sigma: float
    T: float
    barrier: float | None
    is_call: bool
    is_knock_out: bool
```

```
    is_down: bool
    rebate: float = 0.0
    n_steps: int = 500


def quadratic_interp(x, y, x0):
    idx = np.searchsorted(x, x0)
    if idx < 1:
        return y[0]
    elif idx > len(x) - 2:
        return y[-1]
    else:
        x1, x2, x3 = x[idx - 1:idx + 2]
        y1, y2, y3 = y[idx - 1:idx + 2]
        l1 = (x0 - x2)*(x0 - x3) / ((x1 - x2)*(x1 - x3))
        l2 = (x0 - x1)*(x0 - x3) / ((x2 - x1)*(x2 - x3))
        l3 = (x0 - x1)*(x0 - x2) / ((x3 - x1)*(x3 - x2))
        return l1*y1 + l2*y2 + l3*y3


def price_barrier_option(opt: BarrierOption) -> float:
    if not opt.is_knock_out and opt.barrier is not None:
        # Knock-in = vanilla - knock-out
        vanilla = price_barrier_option(BarrierOption(
            S0=opt.S0, X=opt.X, r=opt.r, sigma=opt.sigma, T=opt.T,
            barrier=None,  # vanilla option
```

```
        is_call=opt.is_call, is_knock_out=False, is_down=opt.is_down,
        rebate=opt.rebate, n_steps=opt.n_steps
    ))
    knock_out = price_barrier_option(BarrierOption(
        S0=opt.S0, X=opt.X, r=opt.r, sigma=opt.sigma, T=opt.T,
        barrier=opt.barrier, is_call=opt.is_call, is_knock_out=True,
        is_down=opt.is_down, rebate=opt.rebate, n_steps=opt.n_steps
    ))
    return vanilla - knock_out

dt = opt.T / opt.n_steps
k = np.sqrt(1.5)
dy = k * opt.sigma * np.sqrt(dt)

y0 = np.log(opt.S0)
yb = np.log(opt.barrier) if opt.barrier is not None else y0
padding = abs(y0 - yb) if opt.barrier is not None else 1.0
y_min = min(yb, y0 - 10 * padding)
y_max = y0 + 10 * padding

ny = int(np.round((y_max - y_min) / dy)) + 1
ys = np.linspace(y_min, y_max, ny)

nu = opt.r - 0.5 * opt.sigma**2
edr = np.exp(-opt.r * dt)
pu = 0.5 * dt * ((opt.sigma / dy)**2 + (nu / dy))
```

```
    pd = 0.5 * dt * ((opt.sigma / dy)**2 - (nu / dy))
    pm = 1.0 - dt * (opt.sigma / dy)**2

    ST = np.exp(ys)
    values = np.maximum(ST - opt.X, 0) if opt.is_call else np.maximum(opt.X - ST, 0)

    if opt.is_knock_out and opt.barrier is not None:
        if opt.is_down:
            values[ys < yb] = opt.rebate
        else:
            values[ys > yb] = opt.rebate

    for _ in range(opt.n_steps):
        values[1:-1] = edr * (pu * values[2:] + pm * values[1:-1] + pd * values[:-2])
        if opt.is_knock_out and opt.barrier is not None:
            if opt.is_down:
                values[ys < yb] = opt.rebate
            else:
                values[ys > yb] = opt.rebate

    return quadratic_interp(ys, values, y0)


if __name__ == "__main__":
    for is_call in [True, False]:
        for is_knock_out in [True, False]:
```

```
        opt = BarrierOption(
            S0=100, X=100, r=0.05, sigma=0.179596255695, T=1.0,
            barrier=80, is_call=is_call,
            is_knock_out=is_knock_out, is_down=True,
            rebate=0.0, n_steps=10000
        )
        price = price_barrier_option(opt)
        label = f"{'Call' if is_call else 'Put'} {'KO' if is_knock_out else 'KI'}"
        print(f"{label}: {price:.4f}")

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

#DELTA UP AND OUT CALL

```
#  Paramètres
K = 100
H = 120
r = 0.05
sigma = 0.2
n_sim = 10000
n_steps = 100
S0_range = np.linspace(80, 125, 100)
T_list = [0.25, 0.5, 1.0]

def monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps):
```

```
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma ** 2) * dt +
              sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths >= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, 0, np.maximum(ST - K, 0))
    return np.exp(-r * T) * np.mean(payoff)

def compute_delta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h=0.5):
    V_plus = np.array([monte_carlo_up_and_out_call(S0 + h, K, H, T, r, sigma, n_sim,
n_steps) for S0 in S0_range])
    V_minus = np.array([monte_carlo_up_and_out_call(S0 - h, K, H, T, r, sigma, n_sim,
n_steps) for S0 in S0_range])
    delta = (V_plus - V_minus) / (2 * h)
    return delta

plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Simulation pour T = {T} ...")
    delta_raw = compute_delta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps)
    delta_smooth = savgol_filter(delta_raw, window_length=11, polyorder=3)

    plt.plot(S0_range, delta_smooth, label=f'T = {T:.2f} year')
```

```
plt.axhline(0, color='gray', linestyle='--')
plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Delta Up-and-Out Call")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Delta")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

DELTA DOWN AND IN PUT

```
#  Paramètres
K = 100          # Strike
H = 80           # Barrière down-and-in
r = 0.05         # Taux sans risque
```

```
sigma = 0.2        # Volatilité
n_sim = 10000
n_steps = 100
S0_range = np.linspace(60, 110, 100)
T_list = [0.25, 0.5, 1.0]

def monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma ** 2) * dt +
              sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))

    barrier_hit = np.any(S_paths <= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, np.maximum(K - ST, 0), 0)
    return np.exp(-r * T) * np.mean(payoff)

def compute_delta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h=0.5):
    V_plus = np.array([monte_carlo_down_and_in_put(S0 + h, K, H, T, r, sigma, n_sim,
n_steps) for S0 in S0_range])
    V_minus = np.array([monte_carlo_down_and_in_put(S0 - h, K, H, T, r, sigma, n_sim,
n_steps) for S0 in S0_range])
    delta = (V_plus - V_minus) / (2 * h)
    return delta

plt.figure(figsize=(10, 6))
```

```
for T in T_list:
    print(f"Simulation pour T = {T} ...")
    delta_raw = compute_delta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps)

    delta_smooth = savgol_filter(delta_raw, window_length=11, polyorder=3)

    plt.plot(S0_range, delta_smooth, label=f'T = {T:.2f} year')

#  Graphique
plt.axhline(0, color='gray', linestyle='--')
plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Delta Put Down-and-In ")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Delta")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

from numpy.polynomial.polynomial import Polynomial
```

GAMMA UP AND OUT CALL

```
# Paramètres
```

```
K = 100
H = 120
r = 0.05
sigma = 0.2
n_sim = 10000
n_steps = 100
h = 0.5
S0_range = np.linspace(80, 125, 80)
T_list = [0.25, 0.5, 1.0]

def monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
              sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths >= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, 0, np.maximum(ST - K, 0))
    return np.exp(-r * T) * np.mean(payoff)

def compute_gamma_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h):
    gamma = []
    for S0 in S0_range:
        V_plus = monte_carlo_up_and_out_call(S0 + h, K, H, T, r, sigma, n_sim, n_steps)
        V = monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps)
```

```
        V_minus = monte_carlo_up_and_out_call(S0 - h, K, H, T, r, sigma, n_sim, n_steps)
        g = (V_plus - 2 * V + V_minus) / h**2
        gamma.append(g)
    return np.array(gamma)
plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Calcul gamma pour T = {T}")
    gamma_raw = compute_gamma_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h)

    coeffs = Polynomial.fit(S0_range, gamma_raw, deg=5)
    gamma_fit = coeffs(S0_range)

    plt.plot(S0_range, gamma_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Gamma Up-and-Out Call ")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Gamma")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

GAMMA DOWN AND IN PUT

```
#Paramètres
K = 100
H = 80
r = 0.05
sigma = 0.2
n_sim = 10000
n_steps = 100
h = 0.5
S0_range = np.linspace(60, 110, 80)
T_list = [0.25, 0.5, 1.0]


def monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
              sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths <= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, np.maximum(K - ST, 0), 0)
    return np.exp(-r * T) * np.mean(payoff)


# === Calcul Gamma numérique
def compute_gamma_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h):
    gamma = []
```

```
    for S0 in S0_range:
        V_plus = monte_carlo_down_and_in_put(S0 + h, K, H, T, r, sigma, n_sim, n_steps)
        V = monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps)
        V_minus = monte_carlo_down_and_in_put(S0 - h, K, H, T, r, sigma, n_sim, n_steps)
        g = (V_plus - 2 * V + V_minus) / h**2
        gamma.append(g)
    return np.array(gamma)

#Tracé des courbes
plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Calcul gamma pour T = {T}")
    gamma_raw = compute_gamma_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, h)
    coeffs = Polynomial.fit(S0_range, gamma_raw, deg=5)
    gamma_fit = coeffs(S0_range)

    plt.plot(S0_range, gamma_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Gamma Down-and-In Put ")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Gamma")
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

THETA UP AND OUT CALL

```
#Paramètres
K = 100
H = 120
r = 0.05
sigma = 0.2
n_sim = 10000
n_steps = 100
dT = 1/365  # Variation de temps (~1 jour)
S0_range = np.linspace(80, 125, 80)
T_list = [0.25, 0.5, 1.0]

def monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
              sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths >= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, 0, np.maximum(ST - K, 0))
    return np.exp(-r * T) * np.mean(payoff)
```

```
# Calcul du Theta
def compute_theta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, dT):
    theta = []
    for S0 in S0_range:
        if T - dT <= 0:
            theta.append(0)
            continue
        V_now = monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps)
        V_later = monte_carlo_up_and_out_call(S0, K, H, T - dT, r, sigma, n_sim, n_steps)
        th = (V_later - V_now) / dT  # theta = dV/dT ≈ (V(t+dt) - V(t)) / dt
        theta.append(th)
    return np.array(theta)

#Tracé des courbes
plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Calcul du theta pour T = {T}")
    theta_raw = compute_theta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, dT)
    coeffs = Polynomial.fit(S0_range, theta_raw, deg=5)
    theta_fit = coeffs(S0_range)

    plt.plot(S0_range, theta_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
```

```
plt.title("Theta Up-and-Out Call")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Theta ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

##THETA DOWN AND INT PUT

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial.polynomial import Polynomial

K = 100
H = 80
r = 0.05
sigma = 0.2
n_sim = 10000
n_steps = 100
dT = 1 / 365  # 1 jour
S0_range = np.linspace(60, 110, 80)
T_list = [0.25, 0.5, 1.0]

#  Simulation Monte Carlo pour Down-and-In Put
```

```
def monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
                                    sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths <= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, np.maximum(K - ST, 0), 0)
    return np.exp(-r * T) * np.mean(payoff)

def compute_theta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, dT):
    theta = []
    for S0 in S0_range:
        if T - dT <= 0:
            theta.append(0)
            continue
        V_now = monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps)
        V_later = monte_carlo_down_and_in_put(S0, K, H, T - dT, r, sigma, n_sim, n_steps)
        th = (V_later - V_now) / dT  # Approximation : dV/dT
        theta.append(th)
    return np.array(theta)

# Tracé
plt.figure(figsize=(10, 6))
```

```
for T in T_list:
    print(f"Calcul du theta pour T = {T}")
    theta_raw = compute_theta_mc(S0_range, K, H, T, r, sigma, n_sim, n_steps, dT)

    coeffs = Polynomial.fit(S0_range, theta_raw, deg=5)
    theta_fit = coeffs(S0_range)

    plt.plot(S0_range, theta_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Theta Down-and-In Put")
plt.xlabel("Underlying asset price (S0)")
plt.ylabel("Theta ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#####
#Paramètres
K = 100
H = 120
r = 0.05
sigma = 0.2
dsigma = 0.01
```

```
n_sim = 10000
n_steps = 100
S0_range = np.linspace(80, 125, 80)
T_list = [0.25, 0.5, 1.0]

#
def monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
                                    sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths >= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, 0, np.maximum(ST - K, 0))
    return np.exp(-r * T) * np.mean(payoff)

#
def compute_vega_mc(S0_range, K, H, T, r, sigma, dsigma, n_sim, n_steps):
    vega = []
    for S0 in S0_range:
        V_plus = monte_carlo_up_and_out_call(S0, K, H, T, r, sigma + dsigma, n_sim, n_steps)
        V_minus = monte_carlo_up_and_out_call(S0, K, H, T, r, sigma - dsigma, n_sim,
n_steps)
        v = (V_plus - V_minus) / (2 * dsigma)
        vega.append(v)
    return np.array(vega)
```

```
# Tracé Vega
plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Calcul de Vega pour T = {T}")
    vega_raw = compute_vega_mc(S0_range, K, H, T, r, sigma, dsigma, n_sim, n_steps)

    coeffs = Polynomial.fit(S0_range, vega_raw, deg=5)
    vega_fit = coeffs(S0_range)

    plt.plot(S0_range, vega_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Vega Up-and-Out Call")
plt.xlabel("PUnderlying asset (S0)")
plt.ylabel("Vega ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
##
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial.polynomial import Polynomial

# Paramètres
K = 100
H = 80
r = 0.05
sigma = 0.2
dsigma = 0.01
n_sim = 10000
n_steps = 100
S0_range = np.linspace(60, 110, 80)
T_list = [0.25, 0.5, 1.0]

def monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
```

```
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
                                    sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths <= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, np.maximum(K - ST, 0), 0)
    return np.exp(-r * T) * np.mean(payoff)


def compute_vega_mc(S0_range, K, H, T, r, sigma, dsigma, n_sim, n_steps):
    vega = []
    for S0 in S0_range:
        V_plus = monte_carlo_down_and_in_put(S0, K, H, T, r, sigma + dsigma, n_sim,
n_steps)
        V_minus = monte_carlo_down_and_in_put(S0, K, H, T, r, sigma - dsigma, n_sim,
n_steps)
        v = (V_plus - V_minus) / (2 * dsigma)
        vega.append(v)
    return np.array(vega)


plt.figure(figsize=(10, 6))


for T in T_list:
    print(f"Calcul de Vega pour T = {T}")
    vega_raw = compute_vega_mc(S0_range, K, H, T, r, sigma, dsigma, n_sim, n_steps)
    coeffs = Polynomial.fit(S0_range, vega_raw, deg=5)
    vega_fit = coeffs(S0_range)
```

```
    plt.plot(S0_range, vega_fit, label=f"T = {T:.2f} year")


plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Vega Down-and-In Put")
plt.xlabel("Underlying asset price(S0)")
plt.ylabel("Vega ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

RHO

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial.polynomial import Polynomial

K = 100
H = 120
r = 0.05
dr = 0.01
sigma = 0.2
n_sim = 10000
n_steps = 100
S0_range = np.linspace(80, 125, 80)
```

```
T_list = [0.25, 0.5, 1.0]


def monte_carlo_up_and_out_call(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
                                    sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths >= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, 0, np.maximum(ST - K, 0))
    return np.exp(-r * T) * np.mean(payoff)


def compute_rho_mc(S0_range, K, H, T, r, dr, sigma, n_sim, n_steps):
    rho = []
    for S0 in S0_range:
        V_plus = monte_carlo_up_and_out_call(S0, K, H, T, r + dr, sigma, n_sim, n_steps)
        V_minus = monte_carlo_up_and_out_call(S0, K, H, T, r - dr, sigma, n_sim, n_steps)
        rh = (V_plus - V_minus) / (2 * dr)
        rho.append(rh)
    return np.array(rho)


plt.figure(figsize=(10, 6))


for T in T_list:
    print(f"Calcul de Rho pour T = {T}")
```

```
    rho_raw = compute_rho_mc(S0_range, K, H, T, r, dr, sigma, n_sim, n_steps)

    coeffs = Polynomial.fit(S0_range, rho_raw, deg=5)
    rho_fit = coeffs(S0_range)

    plt.plot(S0_range, rho_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Rho Up-and-Out Call ")
plt.xlabel("Underlying asset price(S0)")
plt.ylabel("Rho ")
plt.legend()
plt.grid(True)


plt.tight_layout()
plt.show()
```

RHO UO CALL

```
K = 100
H = 80
r = 0.05
dr = 0.01
sigma = 0.2
n_sim = 10000
n_steps = 100
```

```
S0_range = np.linspace(60, 110, 80)
T_list = [0.25, 0.5, 1.0]
def monte_carlo_down_and_in_put(S0, K, H, T, r, sigma, n_sim, n_steps):
    dt = T / n_steps
    Z = np.random.randn(n_sim, n_steps)
    S_paths = S0 * np.exp(np.cumsum((r - 0.5 * sigma**2) * dt +
                                    sigma * np.sqrt(dt) * Z, axis=1))
    S_paths = np.hstack((S0 * np.ones((n_sim, 1)), S_paths))
    barrier_hit = np.any(S_paths <= H, axis=1)
    ST = S_paths[:, -1]
    payoff = np.where(barrier_hit, np.maximum(K - ST, 0), 0)
    return np.exp(-r * T) * np.mean(payoff)

def compute_rho_mc(S0_range, K, H, T, r, dr, sigma, n_sim, n_steps):
    rho = []
    for S0 in S0_range:
        V_plus = monte_carlo_down_and_in_put(S0, K, H, T, r + dr, sigma, n_sim, n_steps)
        V_minus = monte_carlo_down_and_in_put(S0, K, H, T, r - dr, sigma, n_sim, n_steps)
        rh = (V_plus - V_minus) / (2 * dr)
        rho.append(rh)
    return np.array(rho)

plt.figure(figsize=(10, 6))

for T in T_list:
    print(f"Calcul de Rho pour T = {T}")
```

```
    rho_raw = compute_rho_mc(S0_range, K, H, T, r, dr, sigma, n_sim, n_steps)
    coeffs = Polynomial.fit(S0_range, rho_raw, deg=5)
    rho_fit = coeffs(S0_range)

    plt.plot(S0_range, rho_fit, label=f"T = {T:.2f} year")

plt.axvline(K, color='yellow', linestyle='--', label='Strike K')
plt.axvline(H, color='red', linestyle='--', label='Barrier H')
plt.title("Rho Down-and-In Put")
plt.xlabel("Underlying asset (S0)")
plt.ylabel("Rho ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

RANDOM PATH UNDERLYING ASSET GRAPHQIUE

```
# Parameters
S0 = 100        # initial asset price
K = 100         # strike price
```

```
H = 120          # barrier level
T = 1.0          # maturity in years
r = 0.05         # risk-free rate
sigma = 0.2      # volatility
n_steps = 252    # daily steps in 1 year
dt = T / n_steps

# Generate one GBM path
np.random.seed(42)
Z = np.random.standard_normal(n_steps)
S = np.zeros(n_steps)
S[0] = S0
for t in range(1, n_steps):
    S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np.sqrt(dt) * Z[t])

# Boundary replication strategy:
vanilla_options = pd.DataFrame({
    'Type': ['Long Call', 'Short Call'],
    'Strike': [K, H],
    'Maturity': [T, T],
    'Position': [1, -1]
})

plt.figure(figsize=(12, 6))
plt.plot(np.linspace(0, T, n_steps), S, label='Asset Price Path', color='blue')
plt.axhline(H, color='red', linestyle='--', label='Barrier (KO Level)')
```

```
plt.axhline(K, color='green', linestyle='--', label='Strike')


plt.title('Up-and-Out Call Option Hedging via Boundary Replication')
plt.xlabel('Time (Years)')
plt.ylabel('Asset Price')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


vanilla_options
```

REPLICATION 6 MOIS

```
import numpy as np
from scipy.stats import norm
from scipy.optimize import root_scalar
from collections import defaultdict
# Paramètres
S0 = 100
K = 100
B = 120
T = 1.0
r = 0.05
q = 0.03
sigma = 0.20
dt = 1/12  # pas de temps (non utilisé ici pour la boucle)
```

```
# Fonction Black-Scholes pour un call européen
def bs_call_price(S, K, T, r, q, sigma):
    if T <= 0:
        return max(S - K, 0)
    d1 = (np.log(S/K) + (r - q + 0.5 * sigma**2)*T)/(sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return S*np.exp(-q*T)*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)

# Valeur théorique de l'up-and-out call
def up_and_out_value(S, t):
    if S >= B:
        return 0
    T_remaining = max(T - t, 0)
    return bs_call_price(S, K, T_remaining, r, q, sigma)

portfolio = [{'strike': K, 'expiry': T, 'quantity': 1.0}]
# Fonction pour la valeur du portefeuille
def portfolio_value(portfolio, S, t):
    total = 0.0
    for opt in portfolio:
        T_rem = max(opt['expiry'] - t, 0)
        total += opt['quantity'] * bs_call_price(S, opt['strike'], T_rem, r, q, sigma)
    return total

# Rebalancement unique à t = 0.5 an
```

```
t_rebal = 0.5

theo_val = up_and_out_value(B, t_rebal)
val_at_barrier = portfolio_value(portfolio, B, t_rebal)
diff = val_at_barrier - theo_val

if abs(diff) > 1e-6:
    expiry = T
    new_strike = B
    def objective(x):
        new_portfolio = portfolio + [{'strike': new_strike, 'expiry': expiry, 'quantity': x}]
        return portfolio_value(new_portfolio, B, t_rebal) - theo_val

    sol = root_scalar(objective, bracket=[-1000, 1000], method='bisect')
    if sol.converged:
        qty = sol.root
        portfolio.append({'strike': new_strike, 'expiry': expiry, 'quantity': qty})

# Affichage
print(f"\nPortefeuille final (1 rebalancement à t={t_rebal}) — Valeur à S = {S0} :\n")
print(f"{'Strike':>10} | {'Maturité':>10} | {'Quantité':>10} | {'Valeur à S=100':>18}")
print("-" * 60)

summary = defaultdict(float)
for opt in portfolio:
    key = (opt['strike'], opt['expiry'])
```

```
    summary[key] += opt['quantity']

for (strike, expiry), qty in sorted(summary.items(), key=lambda x: x[0][1]):
    val = qty * bs_call_price(S0, strike, expiry, r, q, sigma)
    print(f"{strike:10.2f} | {expiry:10.2f} | {qty:10.6f} | {val:18.6f}")

# Calcul valeur totale portefeuille à t=0 et S=S0
val_portfolio = portfolio_value(portfolio, S0, 0)
val_theo = up_and_out_value(S0, 0)
ecart = val_portfolio - val_theo

print("\nValeur totale du portefeuille à t=0, S=100 :", val_portfolio)
print("Valeur théorique up-and-out call à t=0, S=100 :", val_theo)
print("Écart (Portefeuille - Théorique) :", ecart)
```

####ERREUR DE REPLICATION POUR 6 MOIS

```
import matplotlib.pyplot as plt
time_grid = np.linspace(0, T, 100)
errors_vs_time = []
values_theoretical = []
values_portfolio = []

S_eval = B
```

```
# Calcul de l'erreur pour chaque instant
for t in time_grid:
    theo_val = up_and_out_value(S_eval, t)
    port_val = portfolio_value(portfolio, S_eval, t)
    err = port_val - theo_val

    values_theoretical.append(theo_val)
    values_portfolio.append(port_val)
    errors_vs_time.append(err)

# Tracé de l'erreur en fonction du temps
plt.figure(figsize=(10, 6))
plt.plot(time_grid, errors_vs_time, label="Erreur de réplication (S = B)", color='darkblue')
plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.xlabel("Temps (t)")
plt.ylabel("Erreur (Portefeuille - Théorique)")
plt.title("Erreur de réplication en fonction du temps à S = B")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

REPLICATION MENSUEL

```
import numpy as np
from scipy.stats import norm
from scipy.optimize import root_scalar
from collections import defaultdict

S0 = 100
K = 100
B = 120
T = 1.0
r = 0.05
q = 0.03
sigma = 0.20
rebate = 0
dt = 1/12  # rebalancement mensuel

# Fonction Black-Scholes pour un call européen
```

```
def bs_call_price(S, K, T, r, q, sigma):
    if T <= 0:
        return max(S - K, 0)
    d1 = (np.log(S/K) + (r - q + 0.5 * sigma**2)*T)/(sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return S*np.exp(-q*T)*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)

# Valeur théorique de l'up-and-out call
def up_and_out_value(S, t):
    if S >= B:
        return 0
    T_remaining = max(T - t, 0)
    return bs_call_price(S, K, T_remaining, r, q, sigma)

times = np.arange(T, 0, -dt)
portfolio = [{'strike': K, 'expiry': T, 'quantity': 1.0}]

# Fonction pour la valeur du portefeuille
def portfolio_value(portfolio, S, t):
    total = 0.0
    for opt in portfolio:
        T_rem = max(opt['expiry'] - t, 0)
        total += opt['quantity'] * bs_call_price(S, opt['strike'], T_rem, r, q, sigma)
    return total

for t in times[1:]:
```

```
    theo_val = up_and_out_value(B, t)
    val_at_barrier = portfolio_value(portfolio, B, t)
    diff = val_at_barrier - theo_val

    if abs(diff) > 1e-6:
        expiry = round(t + dt, 6)  # maturité résiduelle du call ajouté
        new_strike = B  # pour forcer la barrière
        def objective(x):
            new_portfolio = portfolio + [{'strike': new_strike, 'expiry': expiry, 'quantity': x}]
            return portfolio_value(new_portfolio, B, t) - theo_val

        sol = root_scalar(objective, bracket=[-1000, 1000], method='bisect')
        if sol.converged:
            qty = sol.root
            portfolio.append({'strike': new_strike, 'expiry': expiry, 'quantity': qty})

# Affichage
print(f"\nPortefeuille final (12 rebalancements mensuels) — Valeur à S = {S0} :\n")
print(f"{'Strike':>10} | {'Maturité':>10} | {'Quantité':>10} | {'Valeur à S=100':>18}")
print("-" * 60)

summary = defaultdict(float)
for opt in portfolio:
    key = (opt['strike'], opt['expiry'])
    summary[key] += opt['quantity']
```

```
for (strike, expiry), qty in sorted(summary.items(), key=lambda x: x[0][1]):
    val = qty * bs_call_price(S0, strike, expiry, r, q, sigma)
    print(f"{strike:10.2f} | {expiry:10.2f} | {qty:10.6f} | {val:18.6f}")
```

ERREUR REPLICATION MENSUEL

```
import matplotlib.pyplot as plt
# Recalculer l'erreur à chaque date
times_plot = np.linspace(0, T, 100)
errors = []
rep_values = []
theo_values = []

for t in times_plot:
    V_rep = portfolio_value(portfolio, B, t)
    V_theo = up_and_out_value(B, t)
    errors.append(V_rep - V_theo)
    rep_values.append(V_rep)
    theo_values.append(V_theo)

# Tracé
plt.figure(figsize=(10, 5))
```

```
plt.plot(times_plot, errors, label="Erreur de réplication", color='red', linewidth=2)
plt.axhline(0, linestyle='--', color='gray')
plt.xlabel("Temps (années)")
plt.ylabel("Erreur de réplication (valeur portefeuille - valeur théorique)")
plt.title("Erreur de réplication au point S = Barrière (S = 120)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```